Thrashing Returns: Metastable Failures in Prefix-Cached Agent Serving

Akshat Valse* Ali Jannesari
Department of Computer Science, Iowa State University, Ames, IA, USA
avalse@iastate.edu jannesar@iastate.edu

Abstract

Prefix caching keeps an LLM agent session's KV across the tool calls between turns, so a turn that hits prefills only its new tokens. We show that this can make the engine's high-hit state metastable. On SGLang 0.5.22 with Qwen2.5-7B-Instruct, one A100 80GB and a 300k-token KV pool, closed-population throughput $\mu(n)$ peaks at 0.233 sessions/s at $n = 24$ and falls to all-miss by $n = 48$, while an empty server stayed good in single 3000 s runs at nominal rates up to 2.0 times $\lambda_{\text{ref}} =$ 0.0715 sessions/s, the measured all-miss capacity, and was unstable at 2.5. In a pre-registered paired test at 1.35 and 1.65 λ_{ref}, a burst of 200 sessions trapped the server by the registered rule in 5 of 6 runs, and all 6 had zero cache hits over an evaluation window ending 7500 s after it; without the burst 6 of 6 stayed good, and a client-side cap of 20 sessions recovered 6 of 6. A simulator whose cost model was fitted only to server-log timing predicted this outcome in a commit pushed before submission. vLLM 0.32.0's own curve and edge give the same window and cap, with a steeper cliff; exploratory burst-only runs had zero hits in 6 of 6, and a registered paired test is pending. With the pool SGLang sizes itself (955,757 tokens) the cliff moves out about threefold, the from-empty edge is between 5.5 and 6.5 λ_{ref}, and a second pre-registered test at 2.5 and 4.0 λ_{ref} with a cap of 64 put all 18 runs in their predicted states. Rates are nominal; runs use one model, one GPU type and synthetic sessions with exponential tool times and no shared prefixes. We give a quasi-static model, prove that under request-level FCFS and stated assumptions no online eviction rule lets $\mu(n)$ exceed the all-miss capacity as $n \to \infty$, compare published mechanisms in an earlier, uncalibrated simulator, and give a diagnostic: measure $\mu(n)$ for the workload and cap sessions from it.

1 Introduction

An LLM agent that uses tools sends its serving engine sessions of many turns. Each turn's prompt is the previous context plus the output of the tool just run, and between turns the session is idle while the tool runs. Prefix caching keeps the key–value (KV) state of the previous turn in GPU memory, so a follow-up turn whose prefix is still cached prefills only its new tokens [77]. In our workload a follow-up turn that hits prefills 551 tokens, while a miss prefills the whole prompt, about 10,500 tokens on average; over a session, prefill work with every follow-up hitting is about 16 times smaller than with every turn missing.

We show that this can make the engine's good state, in which almost every follow-up turn hits, *metastable*. Let λ_{miss} be the all-miss capacity, the session throughput of a saturated engine when every turn misses, and λ_{ref} its estimate from saturated runs with caching disabled (0.0715 sessions/s on SGLang with a 300,000-token KV pool). In our measurements an empty server carried nominal arrival rates between 1.25 and 2 times its own λ_{ref} for 3000 s with 91–93% of turns hitting, on SGLang and on vLLM with that pool, and up to 5.5 times λ_{ref} when SGLang sized its own KV pool; each rate is a single run, and all of them used one arrival realization, whose realized rates were 6–13% above nominal. A transient burst admits more sessions than the KV pool can hold once their contexts grow. A session with no running request, whether in a tool call or waiting in the queue, holds no lock on its KV, so admitting waiting work evicts it, oldest first; when its next turn is admitted it misses, the miss costs an order of magnitude more prefill, the queue grows, the gap between a session's turns grows with it, and more idle KV is evicted before its owner returns. Every turn then pays a full prefill, and the backlog does not drain (Fig. 1).

Neither ingredient is new. Throughput that collapses when the working sets of the admitted jobs exceed memory is Denning's thrashing [17, 18], and a bad state that a feedback loop sustains after its trigger is gone is a metastable failure [9]. That KV left idle during tool calls is evicted and costly to recompute motivates Continuum, InferCept, MORI and TokenCake [1, 7, 41, 69]; ThunderAgent and NVIDIA Dynamo already treat agents' KV eviction and re-prefill as working-set thrashing [20, 37], and EfficientAgent applies Denning's working-set principle and load control to the host KV tier of a fixed pool of agents, in which serving latency shifts the timing of later calls [62]. That a thrashing multiprogramming system has unstable regimes is classical [15], and the feedback itself is published for a closed population: for RL rollouts under a concurrency ceiling, Hong et al. report vLLM Router's prefix-cache hit rate falling from 92.4% to 4.5%, describe a loop in which evicted continuations turn later turns into cold prefill and the longer queue and session overlap raise eviction pressure, and interrupt it with a session cap [30]. What we add is that feedback's consequence in an open system, where it runs through the gap between a session's turns: load lengthens that gap, which raises the miss rate, which raises the load. Under

*Corresponding author.

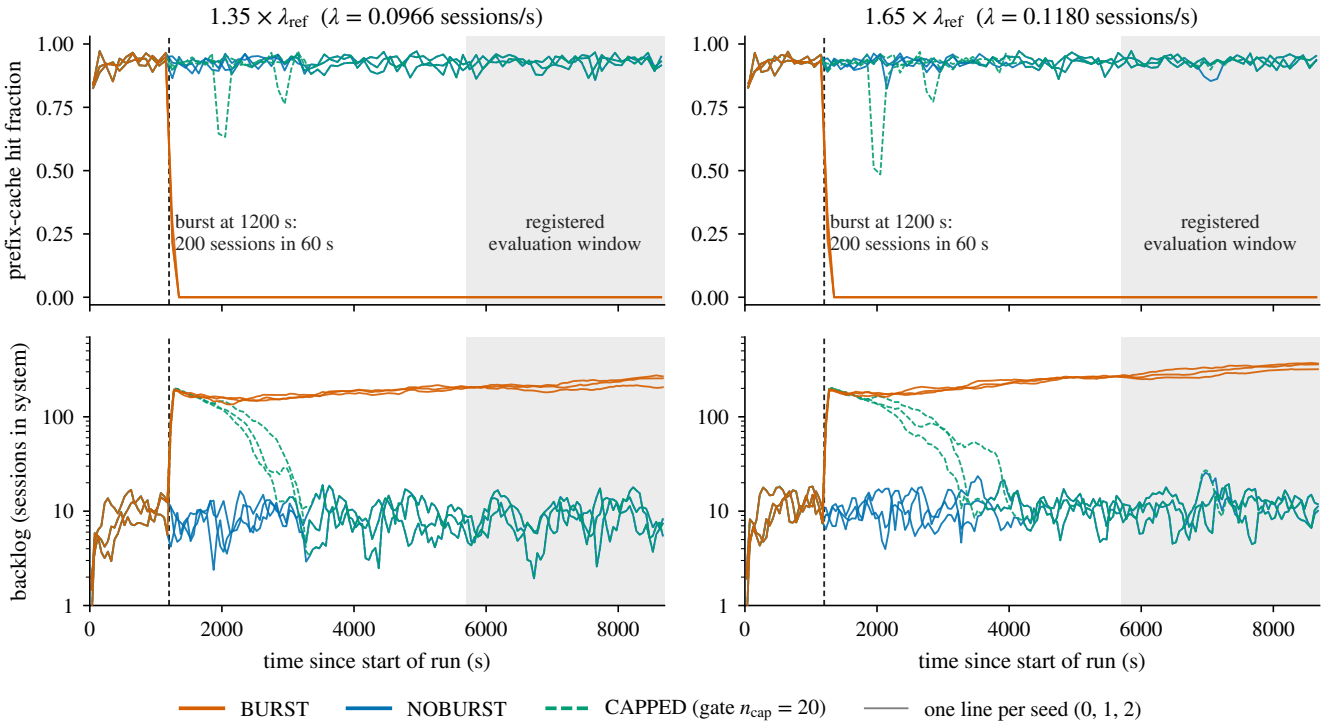


Figure 1: The registered paired test on SGLang (E2) at nominal rates of 1.35 λ_{ref} (left) and 1.65 λ_{ref} (right), λ_{ref} = 0.0715 sessions/s; over the evaluation window the three seeds delivered 1.27–1.45 and 1.61–1.75 λ_{ref}. Top: hit fraction (turns with more than half of the prompt cached) in 100 s windows. Bottom: sessions in the system (log scale), including those in tool calls. One thin line per seed and arm. Dashed: a burst of 200 sessions at 1200 s. Shaded: the registered evaluation window. The hit series is recomputed from the request logs and matches the registered per-run value to four decimals.

the default FCFS ordering, our model and simulator predict that a server that passes steady-state load tests at a rate above λ_{miss} at which it stays good from empty is one sufficiently large burst away from a collapse that, once deep, it does not leave unless the arrival rate falls below λ_{miss} or admission is bounded; no real run lowered the load or imposed a cap after a trap (in the earlier simulator, FCFS recovered from a congested start at 0.9 times its own λ_{miss} but not at 1.0; §8), and on the engine we tested two rates per pool and one burst size. In our earlier simulator, TTL pins with their pinned-first order, under the preview's release rule, and a pure resident-first order (LPM without SGLang's 128-request fallback) raise the rate below which the system recovers (§7). On the engine, stock LPM recovered from a 200-session burst at 1.35 λ_{ref} in 4 of 6 exploratory runs but in only 1 of 6 at 1.65 (§5.6); on our reading of its source, it stops re-sorting its queue when more than 128 requests wait, so in deep congestion Proposition 1 applies to it as well.

Contributions.

- **Two pre-registered real-engine tests** (§4.3) on SGLang 0.5.22 serving Qwen2.5-7B-Instruct on one A100 80GB GPU, each with three paired arms on common random numbers at two registered rates. With a 300,000-token KV pool, at nominal rates of 1.35 and 1.65 λ_{ref}, a burst of 200 sessions trapped the engine by the registered rule in 5 of 6 runs; in the sixth, whose seed delivered the fewest arrivals (1.27 λ_{ref} over the evaluation window), the backlog stayed flat near 206 sessions, and in a post-hoc reading all six were at zero hits from 100 s after the burst to the end of the run. Without the burst the same arrivals stayed good in 6 of 6; with a client-side cap of 20 sessions the burst drained in 6 of 6, at the cost of admission delay at the client (the longest gate wait in each run was 853–985 s; §5.3). With the 955,757-token pool SGLang sizes itself, at 2.5 and 4.0 λ_{ref} with a cap of 64, every run was in the state the registration predicted: trapped in 6 of 6, good in 6 of 6, recovered in 6 of 6 (§5.4). Both pre-registered verdicts are a pass at both rates. Both rates and both pools use the same three seeds, so these runs are not independent across them (§4.1).
- **Measurements that locate the window**: the closed-population thrashing curve $\mu(n)$ on SGLang at two pool sizes and on vLLM 0.32.0 at 300k, λ_{miss} on both engines, and the from-empty stability edge. In nominal rates the edge lies between 2.0 and 2.5 λ_{ref} at 300k on both engines and between 5.5 and 6.5 with SGLang's native pool, in single 3000 s runs that share one arrival realization; at 300k the edge lies below the curve's peak on both engines, while with the native pool the unstable run's arrivals exceeded the peak, so there the data do not separate the edge from the peak. With the native pool the cliff moves out roughly three-

fold, so the 300k-token pool limit did not create it; λ_{miss}, measured with the cache disabled, is unchanged within its standard error. On vLLM the curve peaks at about the same multiple of its own λ_{ref} as on SGLang (3.34 against 3.26) but falls more steeply, to a hit fraction of 0.08 at $n = 32$ against 0.51 (§5.1, §5.2, §5.7).

- **A quasi-static model and blind predictions.** The window is $(\mu_\infty, \min(\mu^*, \lambda_e))$, where μ_∞ is the limit of the closed-population throughput $\mu(n)$ as n grows. Under assumptions stated in §3.4, among them private prefixes, evicted KV that is discarded rather than offloaded, and tool times the engine cannot foresee, we prove that under request-level FCFS no online eviction rule lets μ_∞ exceed λ_{miss} if the congested engine computes prompt tokens no faster than with caching disabled, and that $\mu_\infty = \lambda_{\text{miss}}$ if it computes them as fast; on our reading of its source, the result also covers SGLang's stock LPM, which stops re-sorting its queue when more than 128 requests wait, but not program-level FCFS. Under a session cap c, whenever the gate holds sessions, the admitted ones form a closed c-session system, so in the quasi-static model the congested engine completes sessions at $\mu(c)$, an elementary observation (§3). A simulator whose cost model was fitted only to server-log timing predicted a pass of the first registered test, from a commit pushed before that test was submitted, and the outward shift of the cliff with the native pool before that curve was measured; its from-empty edge was consistent with the real one, but the real from-empty runs had finished before the predictions were committed, so that prediction is blind only by instruction and commit order. A second round, frozen before any native from-empty run had finished and before the native test was submitted, put the native edge at least one grid step too high in nominal multiples in its primary mode, predicted a pass of the native test at rates 6–16% above the registered ones, which it did not simulate, and predicted that the trapped token rate would come close to the rule's tolerance. A third round, pushed before any vLLM curve or from-empty run had finished but after their jobs were submitted, got vLLM's peak level, cap and from-empty edge right ($n = 20$ and 2.0 λ_{ref}) but predicted a later and gentler cliff than vLLM showed: a hit fraction of 0.46–0.53 at $n = 32$ against 0.08 (§5.5, §5.7).
- **Published mechanisms and a diagnostic.** In an earlier, uncalibrated simulator with assumed costs, whose absolute rates do not carry over to the engine, we compare orderings, queue-aware eviction, fixed TTL pins and caps by $\mu(300)$, a proxy for μ_∞; each rule was compared, by reading the code, with released code where it exists (PEEK) and otherwise with the paper (CONCUR), and the pins are configured TTLs, not Continuum's cost model (§7). The memory-demand admission that ThunderAgent and NVIDIA Dynamo implement, MISA-T's adaptive cap and MARS's AIMD window [67] are not evaluated. We give a pre-deployment procedure (§8). Sizing admission from a measured throughput curve is classical load control, and bounding the window by the from-empty edge treats that edge as a finite-horizon stability limit, as Kleinrock and Lam's first exit time does for random-access channels [39]; the procedure's point is that the right cap moves with the tool-time law, the token profile and the pool, so it has to be measured for the workload, and that a cap turns a burst into admission delay at the client.

All real-engine results are for one model, one GPU type and synthetic sessions with exponential 5 s tool times and no shared prefixes. On vLLM 0.32.0 we first ran only the burst arm, at the multiples registered for SGLang and before vLLM's own curve and edge were measured: all 6 runs were at zero hits over the evaluation window, and 4 met the trap rule. These runs are exploratory and do not by themselves show that vLLM's good state is metastable. vLLM's own curve and edge then gave the same window, test points and cap, and a paired test with all three arms was registered from them; the registration was committed before its jobs were submitted but reached GitHub only after the first had started, so it is registered but not pre-registered in our sense, and its BURST arm reuses the exploratory runs' seeds (§5.7). That test was still running when this version was prepared.

2 Background and what is classical

Engines. LLM engines batch per model iteration: running requests decode one token per iteration and waiting requests join between iterations [73]; KV is stored in pages [40]. SGLang keeps cached prefixes in a radix tree, evicts least-recently-used (LRU) leaves, and can order waiting requests by longest prefix match (LPM) [77]. In SGLang 0.5.22 the default policy is FCFS and the default eviction rule LRU; LPM stops sorting its queue when more than 128 requests wait; a waiting request does not lock its cached prefix, which is locked when the request is admitted; and under FCFS, including LPM above that threshold, an LRU refresh touches the prefixes of the first 128 waiting requests once every 32 scheduling passes (files under `python/sglang/srt/` at tag v0.5.22: `arg_groups/fields/schedule.py`, lines 82–98; `arg_groups/fields/memory.py`, lines 25–42; `managers/schedule_policy.py`, lines 133–135, 278–311 and 1591; `mem_cache/radix_cache.py`, lines 552–580; `environ.py`, line 594). vLLM 0.32.0 schedules FCFS, its default, or by priority, and a freed cached block keeps its hash until it is reallocated (`vllm/config/scheduler.py`, lines 21 and 159; `vllm/v1/core/block_pool.py`, lines 666–690 and 774–803; at tag v0.32.0). We first read these sources through a web tool that returns a language model's summary of each file (Appendix D); the line references here and in Appendix E are to the files at the tags, read verbatim. Our synthesis of these rules is that in both engines a session's KV is protected only while one of its requests runs.

Thrashing and metastability. Denning showed that admitting one more program to a memory near saturation can collapse efficiency, and that keeping the working sets of the active programs within memory, which limits their number, prevents it [17, 18]. We call throughput as a function of the closed

population n the thrashing curve; the term is our framing. LRU does badly on loops larger than the cache [13, 63]. A metastable failure is a bad state that persists after its trigger is removed, sustained by a feedback loop under uncontrolled load; one of Bronson et al.'s case studies is a lost look-aside cache [9], and Huang et al. study such failures in production incidents and model them [31]. In Bronson et al.'s terms, a system that serves its load but can be pushed by a trigger into the bad state is in a vulnerable state, and the bad state, kept by a sustaining effect after the trigger ends, is the metastable failure state. We also call the good state metastable, in the physics sense of a state that persists until a large enough perturbation. Gibbens, Hunt and Kelly showed hysteresis in loss networks with random alternative routing; they note that trunk reservation, which refuses alternatively routed calls on a link that is nearly full, removes it [26].

What is classical, and what is specific to agent serving. The collapse past the working-set level, the remedy of limiting active jobs, and a bad state that outlives its trigger are classical; apart from the asymptotic bound of Proposition 1, which is specific to prefix caching, we claim no new queueing theory. The fluid picture of §3.3, a unimodal throughput curve that gives one stable and one unstable equilibrium under open arrivals, is the classical fluid (drift) analysis of unstable random-access channels [24, 39]; network congestion collapse, a stable bad state sustained by retransmissions, is its analogue [33, 51]. Replacing an open system's departure rate by its closed-system throughput is the classical decomposition (flow-equivalent) approximation, whose use to find unstable regimes of thrashing multiprogramming systems goes back to Courtois [15], and choosing the number of admitted jobs from the throughput curve, by paging heuristics [19] or from measured throughput [28, 61], is multiprogramming-level load control. Two features of agent serving make these apply. The reusable KV is exactly the idle KV: a session in a tool call or waiting in the queue holds no running request, so memory-gated admission reclaims its cache. And reuse makes a hit much cheaper than a miss, so losing the cache multiplies the work of every later turn. Both features are the premise of earlier agent-serving systems [1, 7, 8, 41, 69]. ThunderAgent describes agent collapse as working-set thrashing [37], and NVIDIA Dynamo's port of its scheduler aims to avoid thrashing when memory cannot hold every agent's working set [20]; EfficientAgent applies Denning's working-set principle and load control to the host KV tier of a fixed pool of agents and reports that waiting behind other agents stretches the gap between an agent's turns [62]. Hong et al. describe the feedback these features close, as a loop between eviction, cold prefill, queue length and session overlap, for RL rollouts under a concurrency ceiling, and interrupt it with a session cap [30]. What we add is the form it takes through the gap between a session's turns (misses lengthen the queue, the queue lengthens that gap, and longer gaps cause more misses; Proposition 1 formalizes the last link in the large-population limit) and its open-system consequence. We measure that loop on two real engines, locate its window, and compare published mechanisms in these terms in an earlier simulator (§7).

3 Model

3.1 Workload and notation

A session has $T \sim \min(\text{Geometric}(1/15), 60)$ turns, so $E[T] = 14.761$. Turn k sends a prompt of $L_k = L_0 + (k-1)(\Delta + d)$ tokens: a first prompt of $L_0 = 2000$ tokens, $\Delta = 550$ tokens of tool output and $d = 100$ decoded tokens per turn. Between turns the session waits for a tool for a time Z, independent across turns, exponential with mean 5 s in the workload of record. A turn *hits* if more than half of its prompt is served from cache; on the real engine a hit almost always reuses everything but the 550 new tokens and the last decoded token, so it prefills 551 tokens (all but 95 of the 185,613 hits in E2). First turns always miss, so in the long run the hit fraction is at most $1 - 1/E[T] = 0.932$; finite windows can lie slightly above it (up to 0.94 in our runs). With every turn missing, a session's prefill work is $W(T) = 2000\,T + 325\,T(T-1)$ tokens with $E[W] = 154{,}529$; with every follow-up hitting it is $2000 + 551(T-1)$, about 9,600 in expectation.

The engine has a KV pool of M tokens. In the *closed* system n sessions are present and a finished session is replaced at once. The thrashing curve $\mu(n)$ is the long-run session completion rate (sessions/s); $\mu^* = \max_n \mu(n)$ is attained at n^*, and $\mu_\infty = \lim_{n\to\infty} \mu(n)$ is the *congested service rate*, a long-run rate of the closed system (an open system in congestion can complete sessions faster than μ_∞ for a long time, §3.3). The all-miss capacity λ_{miss} is the throughput of the saturated engine with prefix caching disabled. In the *open* system sessions arrive as a Poisson process of rate λ, and $N(t)$ counts sessions in the system (waiting, running or in a tool call). A session rate is estimated either as sessions completed per second or, with much lower variance because session work is heavy-tailed in T, as turns per second divided by $E[T]$ ("turns-based").

3.2 The thrashing curve for agent sessions

Let E_{lb} be the time-averaged KV footprint of a session over its lifetime, a length-biased mean in which long sessions and late turns weigh more (during the tool call after turn k a session holds $L_k + d$ tokens). By the $H = \lambda G$ relation [10, 29], the mean KV demand of n present sessions is nE_{lb}. For our workload E_{lb} is about 10.5k tokens when tool calls dominate the lifetime, so a 300k pool holds $N_{\text{lb}} = M/E_{\text{lb}} \approx 28.5$ sessions on average (weighting every turn equally, the last included, gives 10.6k and 28.4; we set the caps of §7 from the latter). While nE_{lb} is well below M, idle KV survives the tool call and $\mu(n)$ rises with n through batching. Beyond it, admitting a waiting turn evicts idle KV, which belongs to sessions in tool calls or already waiting in the queue; they miss when their next turn is admitted. The peak n^* lies below N_{lb} because resident demand fluctuates around its mean.

3.3 The open system as a quasi-static approximation

Replacing the departure rate of the open system by its closed-system value at the current population, the classical decomposition approximation (§2), gives

$$\dot{n} = \lambda - \mu(n). \tag{1}$$

Extend μ from integer n to all $n \geq 0$ by linear interpolation, so that it is continuous and piecewise linear with $\mu(0) = 0$ and Eq. (1) has unique solutions, and assume that the stationary curve is unimodal: strictly increasing on $[0, n^*]$ to μ^*, and strictly decreasing beyond n^* with limit $\mu_\infty < \mu^*$. For $0 < \lambda < \mu^*$ let $n_1 < n^*$ solve $\mu(n_1) = \lambda$. If $\lambda < \mu_\infty$, n_1 is the only fixed point of Eq. (1), and it attracts every trajectory. If $\mu_\infty < \lambda < \mu^*$, there is one more, $n_2 > n^*$, which is unstable, and from any $n > n_2$, $n \to \infty$: the good state is metastable. If $\lambda > \mu^*$ there is no good state. A solution from a congested state $x_0 > n^*$ returns to n_1 if and only if $\mu > \lambda$ on $(n_1, x_0]$, the endpoint n_1 excluded because $\mu(n_1) = \lambda$; since $\mu > \lambda$ on $(n_1, n^*]$ and $\mu \geq \mu(x_0)$ on $[n^*, x_0]$, this holds if and only if $\lambda < \mu(x_0)$. As x_0 grows, $\mu(x_0)$ falls to μ_∞, so μ_∞ is the *recovery threshold* from deep congestion. After a finite burst x_0 is finite, so recovery above μ_∞ is possible; the threshold for a given burst depends on its size. Without the monotone tail, the recovery threshold from deep congestion is $\inf_{n \geq n^*} \mu(n)$ instead, and congested fixed points can exist (Appendix E proves these statements). Our measured curve is not monotone beyond n^*: in completed sessions it falls to 0.68 λ_{ref} at $n = 48$ and rises to 1.51 λ_{ref} at $n = 300$ (turns-based, 0.88 and 1.56 λ_{ref}; §5.1). Taken at face value, either estimate would put the recovery threshold from $n = 300$ below λ_{miss} and give a stable congested fixed point at 1.35 λ_{ref}. We attribute both features to the synchronized start, which no longer run has tested, and take μ_∞ from Proposition 1.

In a finite system fluctuations can carry N past n_2 without a burst, and an experiment sees only escapes within its horizon. We therefore use a measured *from-empty edge* λ_{e}, the largest rate at which runs started from an empty engine stay stable over a stated horizon, and define the window

$$\mathcal{W} = \big(\mu_\infty,\ \min(\mu^*, \lambda_{\text{e}})\big). \tag{2}$$

Inside $\mathcal{W}$ the good state exists from empty, and a burst that crosses the basin boundary leaves the system in a regime whose service rate is below the arrival rate. That regime is transient (the backlog grows), not a second stationary state, so we say metastable rather than bistable. In the terms of Bronson et al. [9], μ_∞ is the hidden capacity, the load up to which the system heals itself after any trigger, and $\min(\mu^*, \lambda_{\text{e}})$ the advertised capacity; at loads in $\mathcal{W}$ the good state is their vulnerable state, a burst is a trigger whose intensity matters, the congested regime is the metastable failure state, and the feedback of §2, in which misses lengthen the queue and the gap between a session's turns, is its sustaining effect.

Eq. (1) assumes that the internal state (turn-index mix, cache contents, queue order) relaxes quickly compared with changes in n. An automated check of the model by an LLM agent, not independently reviewed (Appendix D), found this reasonable at fixed points and for the sign of the drift in deep congestion, where the stationary μ is flat, but not near the cliff. The model is also not quantitative in congestion, and not only transiently: over the evaluation window the registered SGLang BURST runs still completed 1.13–1.35 λ_{ref} at 300k and 1.44–2.03 times the native λ_{ref}, and their fitted backlog slopes were only 18–69% and 68–78% of the rate $\lambda - \lambda_{\text{miss}}$ that Eq. (1) gives with μ at its congested value λ_{miss}, taking λ as the arrival rate each run received and λ_{miss} as λ_{ref}, or 21–75% and 70–80% with λ_{miss} 3.4% above λ_{ref} (§5.3, §5.4, §3.4). Under FCFS each waiting session advances one turn per cycle of the queue while new sessions keep arriving, so the backlog stays young and its turns are short (in the BURST windows served prompts were 9–20% and 29–41% below the stationary mean, §3.4). At a token rate near the all-miss rate the engine then serves more turns per second than in the stationary mix, and since a session ends after each turn with probability 1/15 until its 60th, it also completes sessions faster than λ_{miss}, for as long as the backlog grows.

A work balance, which we do not make rigorous for the open system, suggests that the recovery threshold depends on the token rate in congestion, not on the departure rate. If hits vanish in congestion (as in every registered SGLang BURST run) and the engine computes prompt tokens at about $C = \lambda_{\text{miss}} E[W]$ (§3.4) while they arrive at $\lambda E[W]$, then for $\lambda > \lambda_{\text{miss}}$ the uncomputed prompt work of the sessions in the system grows, and eventually so does their number, since each holds at most $W(60)$ tokens of it, while for $\lambda < \lambda_{\text{miss}}$ that work shrinks as long as the engine computes at C. If the zero-hit token rate is lower at some population that the system passes through on the way down, the threshold is correspondingly lower; in the fluid model it is $\inf_{n \geq n^*} \mu(n)$ rather than μ_∞. On our engine the closed runs' zero-hit token rate was 4% lower at $n = 48$ than at $n = 96$, with older turns (§3.4). We therefore use Eq. (1) for the window, as a long-run statement for the recovery threshold, and for the capped system's drain time (§3.4), not for burst sizes or backlog growth in congestion: in an earlier simulator with assumed costs (§4.4) the fluid critical burst $n_2 - n_1$ at $2\lambda_{\text{miss}}$ was 20.5 sessions against an empirical 40, because burst sessions are fresh and small.

3.4 Two results

Proposition 1 (Request-level FCFS, any online eviction rule)**.** *Consider the closed system with n sessions under any online eviction rule, one that does not foresee tool times, and assume that its long-run rates exist and are deterministic (Appendix E) and that:*

(A1) sessions share no cached tokens, each has at most one outstanding request, and evicted KV is discarded, not offloaded to another memory tier; tool times are i.i.d. with finite mean, independent of all other randomness, and unknown to the engine until they end;

(A2) admission is request-level FCFS: no request is admitted for the first time before a request that was already waiting

when it joined the queue. More generally, it suffices that the order meets the condition of Appendix E for some Q_0 and S_0 independent of n, which allows reordering while at most Q_0 requests wait and lets a follow-up pass at most S_0 requests that were waiting when it joined. At most B requests run at once, and at most $\alpha t + \beta$ admissions, re-admissions after a preemption included, occur in any interval of length t, with B, α and β independent of n;

(A3) in congestion the engine computes prompt tokens no faster than with caching disabled: $\limsup_{n\to\infty} w_n \le C$, *where w_n is the long-run rate of prompt tokens that are computed rather than served from cache when their request is first admitted (each prompt token is counted once, so tokens recomputed after a preemption are not), and C is the limit of that rate as $n \to \infty$ for the closed system with caching disabled.*

Then the long-run rates of cache-served prompt tokens and of hit turns tend to 0 as $n \to \infty$, so does the hit fraction if $\liminf_{n\to\infty} \mu(n) > 0$, *and* $\limsup_{n\to\infty} \mu(n) \le \lambda_{\text{miss}}$. *If also (A4) $w_n \to C$, that is, the congested engine stays saturated and computes prompt tokens as fast as with caching disabled, then $\mu(n) \to \lambda_{\text{miss}}$, so the hit fraction tends to 0.*

Proof sketch. By (A1), the r_i prompt tokens that follow-up i reuses at its first admission are its session's own and stay in the pool from the end of the session's previous turn until that admission, an interval no shorter than its queue wait D_i, and the token-time that these intervals claim does not overlap. Resident token-time accrues at rate at most M, so follow-ups that wait at least K reuse tokens at rate at most M/K, whatever the eviction rule. By (A2) a follow-up that waits less than K joined behind fewer than $\alpha K + \beta + Q_0 + S_0$ requests, so just before it joined more than $n - B - \alpha K - \beta - Q_0 - S_0$ sessions were in tool calls; Lemma 1 (Appendix E), which uses that the engine does not see tool times, shows that such follow-ups occur at a rate of order $1/n$, and each reuses at most $L_{\max} = 40{,}350$ tokens. Letting $n \to \infty$ and then $K \to \infty$, the cache-served token rate tends to 0, and since a hit reuses more than $L_0/2$ tokens, so does the rate of hits. Counted once, at its request's first admission, each of a session's W prompt tokens is either served from cache or computed, so w_n is $\mu(n)E[W]$ less the cache-served token rate, and with caching disabled the same count gives $C = \lambda_{\text{miss}}E[W]$; (A3) gives the bound and (A4) the limit. The same count gives turns at the rate $\mu(n)E[T]$, so the hit fraction tends to 0 whenever $\mu(n)$ stays bounded away from 0. □

The proof's last step, from vanishing reuse to λ_{miss}, is the bound that MISA-T notes at prefill saturation, a request rate of at most the prefill throughput over the mean uncached prompt tokens per request [30]; what the proposition adds is that request-level FCFS makes reuse vanish in congestion whatever the online eviction rule. (A2) describes an admission pass that stops at the first request that does not fit, as in our simulators and, on our reading of their sources, in SGLang and vLLM under their default FCFS policies (Appendix E); vLLM also preempts running requests and returns them to the head of its queue, which (A2) allows, because its ordering clause concerns first admissions; with asynchronous scheduling it also lets a follow-up pass a bounded number of just-preempted requests, so it meets the general condition of (A2) with $Q_0 = 0$ and a bounded S_0 (Appendix E). A diverging mean cycle time would not be enough: Lemma 1 uses bounded admissions and i.i.d. tool times to bound the tail of the number of sessions in tool calls independently of n, which makes it rare that almost all sessions are in tool calls when one returns. The lemma's bound is of order $1/n$ but has large constants, so the proposition describes the limit, not the populations we ran. The lemma needs FCFS order only while more than Q_0 requests wait, so, on our reading of its source, the proposition also covers SGLang's stock LPM, which stops re-sorting its queue whenever more than 128 requests wait, so that new requests queue behind all waiting ones ($Q_0 = 128$). It does not cover program-level FCFS, which puts an old session's follow-up ahead of newer sessions' requests however long the queue: in our earlier simulator its hit fraction stays near 0.03 from $n = 64$ to 300, while FCFS has none. A pure resident-first order, such as LPM without the fallback, violates (A2) outright: a returning session whose KV is still resident is admitted ahead of the queue.

(A1) holds for our random-token sessions, apart from rare one-token coincidences (5 of the 19,924 first turns in E2 had one cached token), but not for agents that share a system prompt: one resident copy then serves every session. We expect the argument to apply to the private suffixes, with λ_{miss} and C measured with the shared prefix cached, but have not shown this. (A1) also rules out offloading evicted KV to host memory, which would let a follow-up reuse tokens that held no pool space while it waited. (A3) and (A4) treat the engine as a server of prompt tokens, as E0a suggests (§5.2). They concern long-run rates of the closed system, in which every order serves turns in the stationary mix, since each completed session contributes all its turns; our measurement windows did not have that mix, and the token rate varied between them, so we can check (A3) and (A4) only approximately. We estimate C by E0a's uncached token rate at 32 sessions, 11,243 tokens/s. With zero hits, closed runs computed prompt tokens at 1.03 C at $n = 96$, whose served prompts were within 3% of the stationary mean, on SGLang and on vLLM (against vLLM's own C), at 0.99 C at $n = 48$ (prompts 12–14% longer) and at 1.10–1.11 C at $n = 300$ (28–30% shorter); the registered SGLang BURST runs computed 1.01–1.03 C at 300k (prompts 9–20% shorter) and 1.07–1.12 times the 32-session all-miss rate of the native pool (prompts 29–41% shorter). So on our engine (A3) and (A4) hold only approximately: to within a few percent at the stationary mix, and to within 12% over the windows we measured. We estimate λ_{miss} by λ_{ref}, E0a's turns-based rate at 32 sessions (0.0715 sessions/s); λ_{ref} and $C/E[W]$ estimate the same cache-disabled limit. $C/E[W]$ is 0.0728 sessions/s, 1.7% above λ_{ref}, because the turns in E0a's window were longer than the stationary mix (10,648 against 10,469 prompt tokens on average). E0a's 64-session check computed 1.7% more tokens per second than the 32-session runs and 3.2% more than the same seed at 32 sessions, although its window held longer turns (11,456 prompt tokens on average), so

the 32-session reference falls short of saturation by about 2–3%, perhaps more. With the 64-session rate in place of C, $C/E[W]$ is 1.034 λ_{ref}, and with the zero-hit rate at $n = 96$ (11,602 tokens/s) it is 1.050 λ_{ref}. Under (A4) the proposition therefore puts μ_∞ about 2–5% above λ_{ref}, an offset within the standard error of λ_{ref} over seeds (0.0038 sessions/s, 5%); on vLLM the three estimates are 1.020, 1.038 and 1.050 λ_{ref} ($\lambda_{\text{ref}} = 0.0657$ sessions/s).

The bound holds for every online eviction rule, including protection of queued prefixes and TTL pins that keep request-level FCFS order, where (A4) can fail. In our earlier simulator, under FCFS order, a hard pin of queued prefixes gives 0.919 λ_{miss} at $n = 300$ (§7) but 1.04–1.15 λ_{miss} at $n = 48$–150 (3 seeds), as a limit statement allows, and TTL pins of 5–20 s, which that simulator keeps past expiry while the session's next request waits, give 0.10–0.29 λ_{miss} at $n = 300$ with the earlier release rule of §7, with the GPU busy only 8–33% of the time (3 seeds). Both still hit 13% and 10–23% of turns at $n = 300$, so a finite-n hit fraction does not by itself separate covered orders from uncovered ones. The proposition makes their hit rate vanish as n grows, and their hit fraction too unless their throughput tends to 0, as the TTL pins' may; the hard pin keeps the GPU busy, and its hit fraction falls from 0.23 at $n = 48$ to 0.13 at $n = 300$. The TTL rows of §7 that exceed λ_{miss} also serve pinned programs first, an order that (A2) excludes. Belady's offline rule [6] foresees tool times and is outside (A1): the token-time step of the proof covers it, Lemma 1 does not.

Remark (resident-first orders; a conjecture). Under pure LPM (without SGLang's 128-request fallback) in congestion we expect the follow-up hit fraction to tend to $P(Z < R_\infty)$, with R_∞ the congested lifetime of idle KV, so μ_∞^{LPM} exceeds λ_{miss} through tool calls shorter than R_∞; the simulator agrees (§7), but we have no proof.

Proposition 2 (cap identity; elementary)**.** *Let a FIFO gate admit at most c sessions, each holding its slot from admission to the end of its last turn, tool calls included. In the closed system with n sessions the admitted sessions form a closed system of* $\min(n, c)$ *sessions, so* $\mu_{\text{cap}}(n) = \mu(\min(n, c))$.

Proof. When an admitted session finishes, the gate admits its head at once and the replacement joins the gate. □

With $c \le n^*$, μ_{cap} is non-decreasing, so under Eq. (1) the capped system has one attracting state if and only if $\lambda < \mu(c)$. A burst of b sessions that arrives when $n_1 < c$ sessions are present fills the $c - n_1$ free slots at once, so the gate empties after about $(n_1 + b - c)/(\mu(c) - \lambda)$. The registrations' feasibility check used the cruder $b/(\mu(c) - \lambda)$, which ignores the free slots: 1571 s at 1.35 and 1889 s at 1.65 λ_{ref} for the registered test, and 668 s and 1043 s at 2.5 and 4.0 λ_{ref} for the native one. With the populations present at the burst (5–12 sessions at 300k, 11–32 with the native pool), the fluid estimate is 1453–1508 s and 1757–1804 s, and 491–518 s and 840–876 s (compared with the engine in §5.3 and §5.4). This is the standard closed-system equivalence of a system with a bounded multiprogramming level (§2). The stochastic counterpart (positive recurrence for $\lambda < \mu(c)$) should follow from standard arguments; we have not written them out. A cap $c > n^*$ raises the congested level to $\mu(c)$ but leaves the window $(\mu(c), \mu^*)$ open.

4 Methodology

4.1 Real-engine harness

All real-engine runs used one NVIDIA A100 80GB PCIe GPU per job on the Nova SLURM cluster at Iowa State University, with SGLang 0.5.22 (FlashInfer attention, chunked prefill of 8192 tokens, at most 256 running requests, LRU eviction) serving Qwen2.5-7B-Instruct [59]. The KV pool was capped at 300,000 tokens or, in the native-pool runs, sized by SGLang at 955,757 tokens. Decoding was greedy, 100 tokens per turn (Appendix A lists every setting). The vLLM 0.32.0 runs used the same model, GPU, client and workload with a 300,000-token pool (§5.7).

An open-loop client on the same node sends each turn as a completion request whose prompt is a list of token ids: random content from a seeded generator, extended each turn by the token ids the server returned plus 550 new tool tokens, so a hit covers exactly the previous prompt and output; during a tool call the client sleeps. Cached tokens come from the server's usage report, which the engine's counters and log confirmed in the closed-population runs (Stage C, §4.2). Before every run the cache is flushed and checked with a canary: a 1024-token request that reused 1023 tokens before the flush (1008 on vLLM, which caches whole 16-token blocks) must reuse at most 15 after; it reused none. In the cache-off E0a runs the check only confirms that the engine is idle. Each arm of a registered test ran in its own job on a fresh server.

The BURST arm adds 200 fresh sessions within 60 s at $t_b = 1200$ s to the Poisson stream; NOBURST is the same stream without them; CAPPED is BURST plus a client-side FIFO gate admitting at most n_{cap} sessions, each holding its slot until its last turn ends (the gate of Proposition 2). The burst equals 12 to 38 minutes of base arrivals at the registered rates, and its 2000-token first prompts alone (400k tokens) exceed the 300k-token pool. We took its size from our earlier simulator, where bursts of 100 or more trapped every work-conserving policy, so the test asks whether a burst large enough to cross the basin boundary traps the engine, not how likely such a burst is (in the calibrated simulator 25 sessions never trap and 50 trap about half the time, §6). The arms of one seed share arrival times, turn counts and tool times (common random numbers), so BURST differs from NOBURST by the burst and CAPPED from BURST by the gate, up to GPU timing nondeterminism: each arm ran on a fresh server in its own job, and the arms of one seed did not always share a node. Seeds 0, 1 and 2 are also reused across rates, pools, policy cells and engines. For a given seed the 200 burst sessions are identical in every configuration, the arrival times at a lower rate are a subset of those at a higher rate, and the i-th session to arrive has the same turn count, tool times and client-generated

tokens everywhere. Runs that share a seed are therefore not independent across rates, pools or engines, and the BURST arm of the registered vLLM test repeats the workloads of the exploratory vLLM runs.

4.2 Staged protocol

Each stage's inputs came from the previous one (job IDs in Table 3). *Stage 0*, a smoke test of server and client on one GPU, came first; its server logs and those of Stage C calibrated the simulator (§4.4). *Stage C*, the closed-population sweep, measured $\mu(n)$ with all n sessions starting at $t = 0$, one run per n, a 900 s warmup and an 1800 s window. *E0a*, the all-miss calibration, measured λ_{miss} with the radix cache disabled; the reference rate λ_{ref} is its turns-based value, 0.0715 sessions/s, and arrival rates are multiples of it. Each engine and pool has its own λ_{ref} (0.0718 sessions/s with the native pool, 0.0657 on vLLM), and numeric multiples of λ_{miss} in this paper are multiples of that pool's λ_{ref}. *E0b*, the from-empty sweep, ran Poisson arrivals from an empty engine for 3000 s per rate, with arrival seed 0 at every rate; a run is stable if its final queue is below 50 and its queue rises by at most 10 over the last 1500 s. The registration was then filled and committed, and *E2*, the registered paired test, ran the three arms. Afterwards we repeated the sequence with the native pool (Stage C, E0a, E0b, a second registration and *native E2*) and ran an exploratory policy grid. On vLLM we first measured λ_{miss} and ran the BURST arm alone under an exploratory registration, and later ran its own Stage C and E0b, a third registration and its paired test (§5.7).

4.3 Decision rule and registration

Each run of a registered test lasts 8700 s and is judged over $[t_b + 4500, 8700]$ s; in the rule, λ_{miss} is the cell's λ_{ref}. A BURST run is TRAPPED if $\lambda > \lambda_{\text{miss}}$ and (a) its uncached prefill token rate is within ±15% of the cell's E0a all-miss rate (11,243 tokens/s at 300k) and (b) the slope of its backlog exceeds $0.5(\lambda - \lambda_{\text{miss}})$, the slope being the least-squares slope of 10 s bin means of the sessions in the system (arrived and not ended, including any held at the gate) over the window. A run is RECOVERED if over the last 300 s its hit fraction exceeds 0.8 and its outstanding backlog (requests in flight plus sessions held at the gate) is below 20; NOBURST is good if RECOVERED. A run that is neither is DRAINING if its slope is negative and INDETERMINATE otherwise; the labels take precedence in the order trapped, recovered, draining, indeterminate. A rate PASSES if, in at least two of three seeds, BURST is trapped, NOBURST good and CAPPED recovered in that same seed. Criterion (a) uses tokens because a trapped engine with a growing backlog serves mostly young sessions, whose turns are short, so at the all-miss token rate it completes more turns per second than the stationary all-miss mix, and sessions faster than λ_{miss} (§3.3): on traps simulated in our earlier simulator (§4.4), a session-rate test fired on at most 1 of 29 at 2 λ_{miss} and none of 39 at 2.5, while the token test fired on all 84 traps at 1.5–2.5 λ_{miss}.

The registration was filled by formula from committed copies of the Stage C curve, the E0a calibration and the E0b bracket. The window was $(1, 2.0)\,\lambda_{\text{ref}}$, its upper edge E0b's largest all-stable multiple (below $\mu^*/\lambda_{\text{ref}} = 3.26$). The rates followed a rule proposed in the runbook and adopted in the project status notes on 2026-10-22, the day before E0b was submitted (Appendix B): one third and two thirds of the way across the window, rounded to 0.05, which gives 1.35 and 1.65 λ_{ref} (0.0966 and 0.1180 sessions/s). The cap was the smallest n with $\mu(n) \geq 0.95\mu^*$: $n_{\text{cap}} = 20$, since $\mu(20) = 0.224$ is about 4% below $\mu(24) = 0.233$, inside the rule's 5%. The file was committed and reached the project's GitHub repository at 15:09:54 CDT on 2026-10-24, before the E2 jobs were submitted at 15:13 (Table 6). We call a test pre-registered when its registration file was committed and pushed to GitHub before its runs; this is not an entry in a third-party registry. Commit times are set by the committer, so the external record is GitHub's push log, archived as `provenance/github_activity_2026-10.json` (Appendix C). The job script refuses to start unless the file is committed and unmodified and the job's cell, rates, cap, reference rates, hardware and protocol match it, and writes its SHA-256 into every run; the analysis calls a verdict decisive only for the registered cell and rates with every run bound to the file and started after its commit (Appendix B); the label records this binding, not statistical strength, and the analysis checks commit order, not push time, so its own label "pre-registered" does not by itself meet our definition. The native registration was filled and enforced the same way and reached GitHub 10 minutes before its jobs were submitted (§5.4). The vLLM registration was filled from vLLM's own Stage C curve, E0a calibration and E0b bracket, and does not meet the definition: it was committed 37 s before its 18 jobs were submitted, and the cluster's record of the submission names the file but not its hash, but it reached GitHub 2 min after the submission, when two of the jobs had started and none could have ended, so we call this test registered, not pre-registered (§5.7).

4.4 The calibrated simulator

Our discrete-event simulator models a continuous-batching engine: iterations of at most 8192 prefill tokens and 256 sequences; admission reserves a request's end-of-turn KV; idle KV of sessions in a tool call or waiting is evicted whole, least recently used first; waiting requests pin nothing; a finished session's KV is freed at once, an oracle, since a real engine keeps it as evictable cache. An iteration costs a fixed time plus a per-token term plus attention terms. The five cost parameters were fitted only to SGLang's per-batch timing lines in the Stage 0 and Stage C server logs (decode $R^2 = 0.992$, median error 0.75%; prefill $R^2 = 0.998$, 1.3%), never to $\mu(n)$. It runs mixed prefill–decode iterations or, like SGLang, separate iterations with prefill priority; the primary (mixed) mode was the one closer to the real Stage C $\mu(n)$ under a pre-declared metric (mean relative error 0.104 against 0.127), a margin carried by one cell, and both modes' predictions are reported. It does not model partial prefixes, the waiting-prefix refresh or

the overlap scheduler; reserving whole turns at admission is an upper bound on what paged engines hold (in round 5 of the earlier simulator, on-demand allocation with preemption and SGLang-like ratio reservation kept the curve and the trap).

Validation. The declared level test, simulated μ within 15% of the real value in each of 8 Stage C cells, failed in both modes: mixed batching missed one cell, FCFS $n = 48$, by +42% (0.070 against 0.049 sessions/s), separate batching two. The shape was reproduced: the hit fraction crosses 0.5 at $n = 32.5$ against 32.2, the plateau ($n = 16$–24) agrees within 9.2% and $n = 300$ within 2%, and $\mu(300)/\mu^*$ is 0.50 against 0.46. The single real run at $n = 48$ dipped below the simulator's all-miss level; its turns-based estimate, 0.063 sessions/s, is within 15%. Keeping a finished session's KV as evictable cache, a labelled sensitivity arm, left mixed batching the closer mode (0.118 against 0.146) but lowered the plateau (-15.5% at $n = 24$; 5 of 8 cells within 15%). In the earlier simulator the same change moved the orderings and pins of §7 by at most 0.025 λ_{miss} but stalled the CONCUR surrogate at its minimum cap in 12 of 18 cells; the static caps were not rerun.

Blind predictions. The calibrated simulator then predicted, in both modes (mixed primary), B1: λ_{miss} and the all-miss token rate; B2: the fraction of from-empty runs that stay stable at each rate; B3: the E2 outcome at each rate from 0.09 to 0.16 sessions/s; B4: the closed curve with 600k- and 900k-token pools. They were made by LLM coding agents (Claude Sonnet sessions, each in its own copy of the repository at a commit without the E0a or E0b results), which were instructed not to contact the cluster and not to open the E0a or E0b result directories or the primary copy of the repository holding all results; their only real data were the Stage 0 and Stage C runs on which the simulator was calibrated and validated. They were not technically barred: their tool-call transcripts, which are not in the repository, were checked afterwards by the orchestrating Claude Code session, which found no cluster command and no access to E0b before the predictions reached GitHub. They reached GitHub at 14:53 CDT on 2026-10-24, before any E2 job was submitted and before any native-pool run, so B3 and B4 are blind in time. E0b had already finished on the cluster, and its results were pulled 13 minutes later, so B2 is blind only by instruction and commit order (Table 6). B1 is not blind in time (E0a had been committed), and it is close to the calibration data: the cost model was fitted to Stage C per-batch prefill timing, and the all-miss capacity is essentially a prefill-token rate, so B1 checks the simulator's bookkeeping more than its predictive power. An independent verifier, a separate OpenAI Codex session that reused the prediction agents' copy of the repository rather than a fresh one, reran a subset, whose outputs matched exactly, and recomputed every B2 and B3 classification (Appendix C). A second round, B1n–B4n, repeated the four predictions with the native pool, ten seeds for B2n and B3n. B3n is blind in time; B2n was pushed before any of its runs had finished, but after their jobs were submitted; B1n and B4n are not blind, because the native curve and λ_{miss} were already in the project status notes when the round was specified. No independent verifier reran this round (§5.5, Appendix C). A third round, made by the same kind of agent with the simulator unchanged and with its token costs scaled to vLLM's measured all-miss token rate (a fit, not a prediction), predicted vLLM's closed curve (B4v), from-empty edge (B2v) and paired test (B3v); it reached GitHub 22 minutes after the vLLM Stage C and E0b jobs were submitted and before any of them had ended, and before the vLLM registration existed (§5.7).

Our earlier simulator (rounds 1–5), which is not calibrated to the engine and uses assumed costs (all-miss capacity 0.0474 sessions/s), is used for the mechanism comparison of §7, in units of its own λ_{miss}, and in §3.3, §4.3 and §4.4 where stated.

5 Results on the real engine

5.1 The thrashing curve at two pool sizes

With the 300k pool (Fig. 2, left), $\mu(n)$ rises from 0.176 sessions/s at $n = 16$ to 0.224 at 20 and 0.233 at 24, with hit fractions 0.93, 0.93 and 0.89. The run at $n = 24$, which sets μ^*, contains a degraded episode inside its window (hit fraction 0.66 over $[900, 1200)$ s, 0.46 in its worst 100 s); over $[1200, 2700)$ s alone $\mu(24) = 0.257$ against $\mu(20) = 0.231$, so the near-tie that sets the cap of 20 owes part of its closeness to this episode. The run at $n = 32$ mixes two regimes: degraded from about 450 to 2400 s, then recovered to 0.80, for a hit fraction of 0.51. At $n = 48$ none of the 20.0M prompt tokens in the window was served from cache: evictable tokens survive about 6.3 s before eviction (estimated as mean evictable KV over the eviction rate) while a session returns after 30.6 s on average (2.9 s against 66 s at $n = 96$). The cliff is where memory predicts ($N_{\text{lb}} \approx 28.5$; the hit fraction crosses 0.5 at $n = 32.2$). Token-level hit rates have been reported to collapse as concurrent agents grow on SGLang [12] and on vLLM [30, 75]; what we add is $\mu(n)$ in completed sessions per second, the quantity the open-system model uses, at two pool sizes and, in §5.7, on a second engine. Beyond the peak the curve is not monotone: it falls to $\mu(48) = 0.049$ sessions/s and rises again, $\mu(96) = 0.061 < \mu(300) = 0.108$ sessions/s. We attribute both to the synchronized start, which leaves old, long turns in the window at $n = 48$ (served prompts 14% above the stationary mean) and young sessions with short prompts at $n = 300$ (71% of the stationary length in flight); no longer run has tested this; the all-miss level implied by token throughput is roughly flat at 0.072–0.081 sessions/s, so the collapse from the peak is about threefold. Stock LPM keeps 39% of turns hitting at $n = 48$ (rising from 0.28 to 0.44 through the window); at $n = 300$ its queue of 269 exceeds the 128-request threshold above which, by the source, SGLang's LPM stops re-sorting its queue, and no turn hits; whether the fallback causes the zero was not tested (at $n = 300$ the pool may be oversubscribed under any ordering).

With the native pool (Fig. 2, middle), SGLang sized the KV pool at 955,757 tokens, 3.19 times the 300k pool. μ peaks at 0.479 sessions/s at $n = 64$ (0.477 at 80) with 92–93% of turns

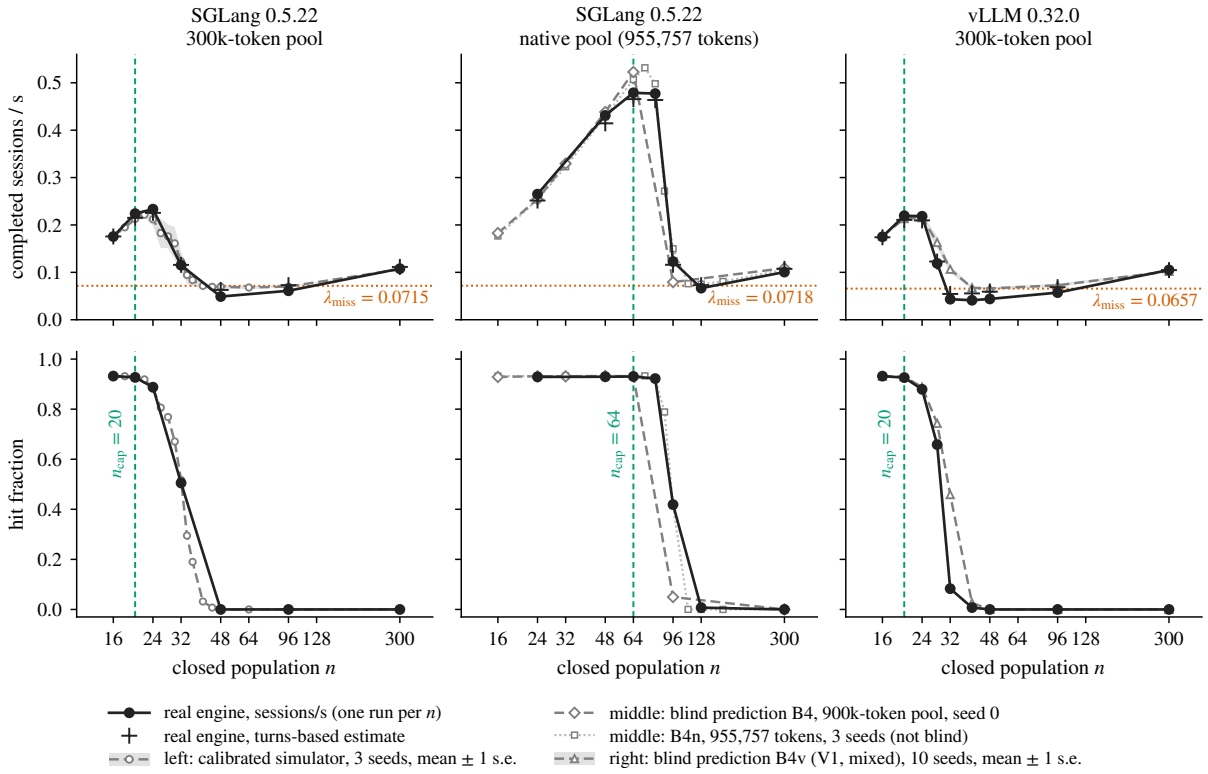


Figure 2: Closed-population thrashing curve, real engine (black, filled; one run per n) against the simulator (gray, open, dashed). Top: $\mu(n)$ in completed sessions/s (plus signs: turns-based estimate); bottom: hit fraction. Left: SGLang, 300k-token pool; calibrated simulator, mean of 3 seeds ±1 s.e. Middle: SGLang, native pool (955,757 tokens); blind prediction B4 at 900k tokens (seed 0); light gray, dotted: B4n at the real pool (3-seed mean), which is not blind (§5.5). Right: vLLM 0.32.0, 300k-token pool; blind prediction B4v (variant V1: token costs scaled to vLLM's all-miss rate; mixed batching), mean of 10 seeds ±1 s.e. (§5.7). Dotted: each cell's λ_{miss}. Green dashed: the registered caps, n = 20, 64 and 20.

hitting, then falls to 0.123 at n = 96 (hit fraction 0.42), 0.067 at 128 (0.007) and 0.101 at 300. The hit fraction crosses 0.5 between n = 80 (0.92) and 96 (0.42), near 93 by linear interpolation, against just above 32 at 300k; the peak is at 64–80 against 20–24. With one run per n, the cliff moves out roughly threefold. The all-miss capacity is unchanged within its standard error: 0.0718 ± 0.0037 sessions/s turns-based against 0.0715 ± 0.0038, and 11,282 ± 51 against 11,243 ± 84 all-miss tokens/s. The 300k-token pool limit did not create the cliff; it moved it. The ratio $\mu^*/\lambda_{\text{ref}}$ is 6.67 with the native pool and 3.26 at 300k.[1]

5.2 The all-miss capacity and the from-empty edge

E0a gives λ_{ref} = 0.0715 ± 0.0038 sessions/s turns-based (standard error over three seeds), our estimate of λ_{miss}, 0.0717 ± 0.0076 in completed sessions, and 11,243 ± 84 all-miss prefill tokens/s. The token rate varies by ±1.2% across seeds and the session rate by ±17%: the engine is bound by prefill tokens, and the sessions completed in 1800 s are a small sample of a heavy-tailed quantity. The token rate divided by $E[W]$ gives 0.0728 sessions/s, within 2% of the turns-based value.

From an empty cache and queue (Fig. 3, top) the engine stayed stable at 1.25, 1.5, 1.75 and 2.0 λ_{ref}, ending each run with an empty queue and with 91–93% of turns hitting over the run, and was unstable at 2.5 λ_{ref} (final queue 127, rising by 146 over the last 1500 s) and 3.0 (289 and 196). The edge lies between 2.0 and 2.5 λ_{ref}, well below $\mu^*/\lambda_{\text{ref}}$ = 3.26, so the from-empty edge, not the peak, bounds the window. The registered window was (1, 2.0) λ_{ref} = (0.0715, 0.143) sessions/s. Each rate is one 3000 s run; a longer horizon could find a lower edge, and at 2.0 λ_{ref} the hit fraction fell from 0.934 in [300, 1500) s to 0.903 in the second half of the run. All six runs used seed 0, so they share one random-number stream: their arrival sequences are nested, and each arrival count lies about 2.1 standard deviations above its Poisson mean, 8–13% above the nominal rate (302, 359, 416, 472, 585 and 697 sessions; the runs labelled 2.0 and 2.5 λ_{ref} received 2.20 and 2.73 λ_{ref}). This makes the stable side of the bracket conservative and its unstable side weaker than the labels suggest; in realized arrivals the bracket is still below $\mu^*/\lambda_{\text{ref}}$.

With the native pool (Fig. 3, middle; native E0b, rates in

[1] These are the registered values: the peak in completed sessions over the turns-based λ_{ref}. Turns-based over turns-based gives 6.48 and 3.15.

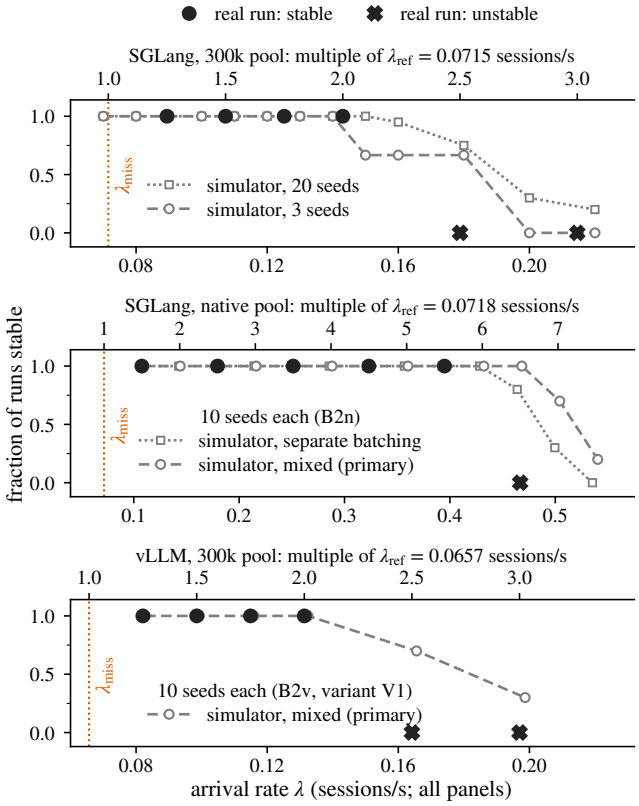


Figure 3: From-empty stability. Real engine: one 3000 s run per rate from an empty cache and queue (1 = stable, 0 = unstable), plotted at its nominal rate; all runs used arrival seed 0, whose realized rates were 6–13% higher. Gray: blind simulator predictions, the fraction of simulated runs stable at multiples of the simulator's own λ_{miss}. Top, SGLang with the 300k pool: B2 (3 seeds dashed, 20 dotted). Middle, SGLang with the native pool: B2n (10 seeds; mixed batching, the primary mode, dashed; separate, dotted). Bottom, vLLM 0.32.0 with the 300k pool: B2v (variant V1, costs scaled to vLLM; mixed batching, 10 seeds). Top axes: multiples of each cell's λ_{ref} (the simulators' own λ_{miss} are 0.0722 at 300k, 0.0720 and 0.0714 native, and 0.0662 for B2v).

multiples of the native λ_{ref} = 0.0718 sessions/s) the engine stayed stable at 1.5, 2.5, 3.5, 4.5 and 5.5 λ_{ref}, ending each run with an empty queue and with 93% of turns hitting over the run, and was unstable at 6.5 λ_{ref}: its final queue was 505, rising by 477 over the last 1500 s, and its hit fraction fell from 0.84 in [300, 1500) s to 0.00 in the second half. In nominal rates the edge lies between 5.5 and 6.5 λ_{ref} (0.395 and 0.467 sessions/s), so the native window is (1, 5.5) λ_{ref}. Unlike at 300k, this does not show the edge below the peak (μ^*/λ_{ref} = 6.67, or 6.48 turns-based): these runs also used seed 0 (361 to 1478 sessions, 5.6–11.7% above nominal), and the 6.5 run received 0.493 sessions/s (6.86 λ_{ref}), above the measured peak, so overload alone can explain its instability. The stable 5.5 run received 0.419 sessions/s (5.84 λ_{ref}). Its upper edge is 2.75 times higher than at 300k in units of λ_{ref} (5.5 against 2.0), and the window is 4.5 λ_{ref} wide against 1.0.

Table 1: The two pre-registered SGLang tests: E2 (300k-token pool, cap 20, λ_{ref} = 0.0715 sessions/s) and native E2 (955,757-token pool, cap 64, λ_{ref} = 0.0718 sessions/s). States for seeds 0, 1, 2: T trapped, I indeterminate, G good, R recovered. Hit fraction, uncached prefill rate in multiples of the all-miss rate, mean backlog (sessions in the system) and its slope (sessions/s) over the evaluation window; ranges over seeds; good-arm slopes in absolute value. Slope thresholds: 0.0125, 0.0232, 0.0538 and 0.1077 at 1.35, 1.65, 2.5 and 4.0 λ_{ref}. Rates are nominal; the base arrivals realized in the window, which the three arms of a seed share, were 1.45, 1.27, 1.42; 1.73, 1.61, 1.75; 2.73, 2.48, 2.58; and 4.28, 3.89, 4.09 λ_{ref} for seeds 0, 1, 2.

λ/λ_{ref}	arm	states	hit	tokens	backlog	slope
300k-token pool (E2)						
1.35	BURST	T I T	0.000	1.009–1.025	200–232	0.0036–0.0222
	NOBURST	G G G	0.930–0.934	0.08–0.09	7.9–9.7	<0.001
	CAPPED	R R R	0.930–0.934	0.08–0.09	7.9–9.7	<0.001
1.65	BURST	T T T	0.000	1.025–1.029	283–332	0.0277–0.0362
	NOBURST	G G G	0.921–0.934	0.10–0.13	10.9–11.8	<0.002
	CAPPED	R R R	0.929–0.934	0.10–0.11	10.9–11.7	<0.002
native pool (native E2)						
2.5	BURST	T T T	0.000	1.074–1.078	557–601	0.079–0.085
	NOBURST	G G G	0.929–0.935	0.15–0.16	15.6–18.5	<0.003
	CAPPED	R R R	0.929–0.935	0.15–0.16	15.6–18.5	<0.003
4.0	BURST	T T T	0.000	1.106–1.124	1098–1139	0.146–0.164
	NOBURST	G G G	0.929–0.935	0.24–0.27	28.2–33.6	<0.004
	CAPPED	R R R	0.929–0.935	0.24–0.27	28.1–33.5	<0.004

Again each rate is a single 3000 s run.

5.3 The registered paired test

The registered analysis, run on all 18 runs with the code frozen at the registration commit, reports a pre-registered, decisive pass at both rates (Table 1, top). Every BURST run had a hit fraction of 0.000 over the evaluation window, an uncached prefill rate of 1.009–1.029 times the all-miss rate and a mean backlog of 200–332 sessions. One (1.35 λ_{ref}, seed 1) is stuck in the same way, but its backlog was flat over the window (207 sessions at its start, 206 at its end; slope 0.0036 sessions/s, below the 0.0125 threshold), so the rule calls it indeterminate: its seed delivered 0.0910 arrivals/s in the window, 5.8% below the nominal 0.0966 (seeds 0 and 2: 0.1037 and 0.1017), and the trapped engine completed 0.0913 sessions/s. The pass at 1.35 λ_{ref} therefore rests on 2 of 3 seeds, the minimum the rule allows. The other runs' base arrivals over the window came to 1.42 and 1.45 λ_{ref} at 1.35 and 1.61–1.75 λ_{ref} at 1.65 (Table 1). Every NOBURST and CAPPED run is good (hit fractions 0.921–0.934, backlogs of 8–12 sessions, most asleep in tool calls); in the window CAPPED cannot be told apart from NOBURST, because the gate had drained the burst. The counts are small, and pooling the two rates does not make them independent: both rates use the same three seeds, with the same burst and nested arrivals (§4.1). BURST's hit fraction is below NOBURST's in all six seed-and-rate pairs; counting each seed once, a post-hoc one-sided sign test gives $p = 1/8$, the smallest value three seeds allow.

In Fig. 1, every BURST run is at zero hits from the window starting at 1300 s to the end. Its backlog first fell, to 135–172 sessions 300–1300 s after the burst, as short burst sessions finished, then rose; over the last 300 s it was 206–369 sessions, still growing in five runs and flat in the sixth. The five trapped runs still completed 0.081–0.097 sessions/s (1.13–1.35 λ_{ref}), above λ_{miss}, because the backlog's sessions are young and their turns short, so that at about the all-miss token rate more turns, and more sessions, complete (§3.3), but below the arrival rate, and the sixth completed sessions about as fast as they arrived (1.28 and 1.27 λ_{ref}); only the uncached prefill rate sits at the all-miss level, which is why the registered rule uses tokens. The CAPPED gate's queue emptied 1547–1904 s after the burst at 1.35 λ_{ref} and 2021–2693 s at 1.65 λ_{ref}, 3–52% longer than the fluid estimate of §3.4 from the population present at the burst (1453–1508 s and 1757–1804 s; the registration's cruder estimate was 1571 s and 1889 s), and well before the evaluation window. Outside the registered criteria, the CAPPED arm's hit fraction dips while the gate drains the burst: its lowest 100 s window is 0.48–0.77 in four of six runs, against 0.82–0.89 for NOBURST, which plausibly slowed the drain. The cap has a cost that the registered criteria do not measure: it turns the burst into admission delay at the client. In each CAPPED run 349–543 sessions waited at the gate, 45–66% of them ordinary arrivals queued behind the burst, for 405–471 s on average and at most 853–985 s; outside the drain the gate bound only briefly, when arrivals clustered (at most 40 sessions in a run, each for less than 50 s). Without the cap, turns completed in the BURST arm's evaluation window took a mean of 163–237 s from send to completion (the client does not stream), against 1.4–1.7 s in NOBURST and CAPPED and 1.4–1.5 s in every arm before the burst; at 8700 s, 19–29% of BURST sessions were unfinished. So at loads that the engine carried from empty for the full 8700 s run (E0b carried the bracketing rates, 1.25–1.75 λ_{ref}, for 3000 s), 200 sessions arriving within a minute, an order of magnitude more than the 8–12 present in the good state, moved it into an all-miss state that it did not leave within 7500 s, and a cap of 20 sessions, within 5% of the measured peak of $\mu(n)$ (argmax $n = 24$), prevented that at the price of that admission delay.

5.4 The registered test with the native pool

A second registration was filled by the same formulas from the native curve, the native E0a calibration and the native E0b bracket, and committed before the first native E2 job (Appendix B). Its window is (1, 5.5) λ_{ref}; the one-third/two-thirds rule, written down before native E0b was read, gives 2.5 and 4.0 λ_{ref} (0.1795 and 0.2872 sessions/s); the cap rule gives $n_{\text{cap}} = 64$, the peak (flagged ambiguous: $\mu(80)$ is within 0.35% of $\mu(64)$). The protocol, the burst of 200 sessions and the trap rule are E2's, with the native reference rates.

The registered analysis reports a pre-registered, decisive pass at both rates, and every run is in the predicted state (Table 1, bottom; Fig. 4). All six BURST runs were trapped: hit fraction 0.000 over the evaluation window, an uncached prefill rate of 1.074–1.078 (2.5 λ_{ref}) and 1.106–1.124 (4.0 λ_{ref}) times the native all-miss rate of 11,282 tokens/s, and mean backlogs of 557–601 and 1098–1139 sessions with slopes above the threshold. All six NOBURST runs were good and all six CAPPED runs recovered (hit fractions 0.929–0.935, backlogs of 16–34 sessions, most asleep in tool calls). Over the window the base arrivals were 2.48–2.73 λ_{ref} at 2.5 and 3.89–4.28 at 4.0. Counting each seed once, the post-hoc sign test again gives $p = 1/8$. Pooled post hoc over the two pre-registered SGLang tests (a descriptive count, not a registered quantity: the tests differ in pool, cap and rates, and reuse the same three seeds, so the 12 runs of each arm share three seeds' session sequences), NOBURST was good and CAPPED recovered in 12 of 12 runs each, and BURST was at zero hits in 12 of 12 and trapped by the rule in 11 of 12.

With the larger pool the collapse took longer and the backlog never drained (Fig. 4). In the 100 s window that contains the burst, the native BURST runs still hit with 0.99–1.00 of their follow-up turns (all but 4 of 3,326), against 0.43–0.59 at 300k; counting the burst's own first turns, which always miss, they hit with 0.63–0.76 of all turns (0.21–0.33 at 300k). Every BURST run was at zero hits from 300–600 s after the burst to the end of the run. In 50 s bins its backlog fell after the burst only at 2.5 λ_{ref}, to 155–168 sessions 150–200 s after it; at 4.0 λ_{ref} it grew from the first bin after the burst (201–231 sessions). It then grew to 658–1379 sessions over the last 300 s. The trapped engine completed 0.103–0.146 sessions/s (1.44–2.03 λ_{ref}), above λ_{miss}, because its backlog's sessions are young and their turns short (§3.3), and well below the arrival rates; it prefilled 1.074–1.124 times the 32-session all-miss token rate, as the simulator predicted (§5.5). The CAPPED gate's queue emptied 512–583 s after the burst at 2.5 λ_{ref} and 837–937 s at 4.0, between 4% shorter and 13% longer than the fluid estimate from the population present at the burst (491–518 s and 840–876 s), and 10–23% shorter than the registration's cruder 668 s and 1043 s. Here 234–448 sessions waited at the gate, 128–180 s on average and at most 247–361 s; outside the drain the gate bound for at most 6 sessions in a run, each for at most 7.1 s. BURST turns completed in the evaluation window took 321–511 s on average, against 1.6–2.5 s in NOBURST and CAPPED, and 38–50% of BURST sessions were unfinished at 8700 s. Here 200 sessions were 6 to 13 times the 16–34 present in the good state. B3n had predicted a pass at somewhat higher rates (§5.5).

5.5 Blind predictions against outcomes

B1 matched within 3.1% (Table 2), which mostly checks the simulator's bookkeeping (§4.4). B2's edge, 0.14–0.15 sessions/s, is consistent with the real edge in [0.143, 0.179); near 0.179 the model keeps 2 of 3 runs stable (15 of 20 with more seeds), so one unstable run there is no contradiction. B3 predicted a pass at every tested rate in both batching modes. At the simulated rates that bracket each registered rate, the primary (mixed) mode's trapped token ratios, 1.003–1.027 at 0.09–0.10 sessions/s and 1.025–1.043 at 0.11–0.12, contain the observed 1.009–1.025 at 0.0966 and 1.025–1.029 at 0.1180; but at 0.12, the grid point nearest the higher

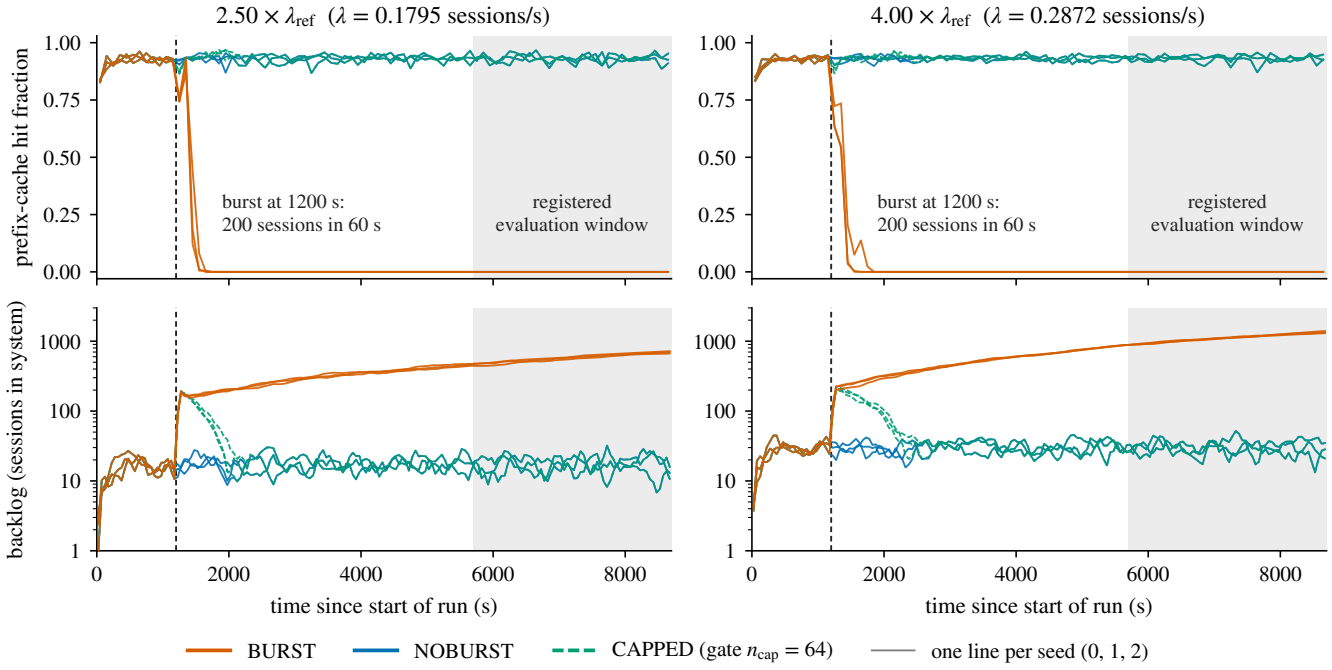


Figure 4: The registered test with the native pool (native E2) at 2.5 λ_{ref} (left) and 4.0 λ_{ref} (right), $\lambda_{\text{ref}} = 0.0718$ sessions/s; layout as in Fig. 1, with the CAPPED gate at $n_{\text{cap}} = 64$ and the backlog axis extended to 3000.

rate, the prediction was 1.034–1.043, above what the engine did. The observed good-arm hit fractions (0.921–0.934) are close to, and at the low end partly below, the predicted 0.929–0.937. B3 flagged both ends of its grid as fragile. At the low end the slope margin $0.5(\lambda - \lambda_{\text{miss}})$ is small: with its registered three seeds it trapped 3 of 3 at both 0.09 and 0.10, but with 20 seeds 14 and 18, so 10–30% of runs there are stuck without meeting the slope test; one indeterminate real run in three at 0.0966 is consistent with that. At 0.16 the 20-seed supplement of the secondary (separate) mode failed the pass rule, with NOBURST good in 13 of 20 seeds, one short; the engine was not run there. (Backlogs compare only roughly: the simulator counts waiting requests, the engine runs outstanding ones.) B4 got the peak's position, within its coarse grid, and the outward shift of the cliff; the predicted peak was 9% above the real one and the cliff early: on B4's grid of 64 and 96 the real hit fraction crosses 0.5 near 91, against B4's 80, and at $n = 96$ the engine still hit 42% of turns, against B4's 5%. The early cliff is mostly the pool: B4n, the same simulator at the real pool (not blind), hit 0.42 at $n = 96$ and crossed 0.5 near 94. The peak error is not: at $n = 64$, where nothing is evicted, seed 0 gives the same 0.523 in both pools. Two qualifications: the verifier's strict check that every B3 pass lies inside the B2 all-stable range failed (B3 asks for 2 of 3 seeds over a longer run), and B2 is blind only by instruction (checked afterwards from the agents' transcripts) and by commit order, not by event order: E0b had finished on the cluster before the predictions were committed, and was pulled 13 minutes after their commit reached GitHub.

A second round (B1n–B4n) reran the same simulator with only the pool changed to 955,757 tokens. B2n and B3n were committed and pushed before any native E0b run had finished, 23 minutes after the native E0b jobs were submitted, and before any native E2 job existed, so B3n is blind in time and B2n is blind in that no result existed, not in submission order; their specification, like that of B1n and B4n, was written after the native curve was known (below), and B3n's cap of 64 is B4n's output. B2n held in part. Both modes predicted stability at all five rates where the engine was stable (10 of 10 seeds each), but the primary (mixed) mode put the all-stable edge at 6.5 and kept 10 of 10 runs stable there, where the real run was unstable, so its edge is too high by at least one step of its 0.5 grid in nominal multiples (the real 6.5 run's arrivals came to 6.86 λ_{ref}, above the real peak, and B4n puts the simulated peak 11% above the real one); the secondary (separate) mode put the edge at 6.0, with 8 of 10 runs stable at 6.5, consistent with the real bracket. Because the real edge came in lower, the registered rates (2.5 and 4.0) lie below B3n's own test points, which are 6–16% higher in multiples of λ_{miss}, and B3n was not simulated at them; 2.5 lies below every point it ran, toward the low end that it had flagged as its weak spot (9 of 10 seeds trapped at 2.85, the tenth recovering). The pass held at both registered rates. The levels are consistent with the lower rates, not a direct match: the trapped token rate rises with the rate in both the simulator and the engine, and the real 1.074–1.078 at 2.5 λ_{ref} lies just below the 1.086–1.109 predicted at 2.65–2.85, while the real 1.106–1.124 at 4.0 overlaps the low end of the 1.119–1.140 predicted at 4.35–4.65. Outstanding backlogs of 549–594 and 1089–1130 requests lie inside the predicted 460–643 and 1067–1333 (they compare only roughly, as for

Table 2: Blind predictions (calibrated simulator, mixed batching unless noted) against outcomes; rates in sessions/s; native and vLLM rates in multiples of λ_{miss} (for the simulator its own, for the engine that cell's λ_{ref}). Percentages in the last column are predicted/real −1. B1 is not blind in time and is close to the calibration data (§4.4, Appendix C). B3 and B3n used a per-arm pass rule (each arm in 2 of 3, resp. 7 of 10, seeds); the registered rule asks for all three arms in the same seed, which B3n's runs also meet (9 of 10 seeds at its low mixed point, 10 of 10 elsewhere). B3's token-ratio and backlog ranges are over the two simulated rates that bracket each registered rate. B4 is the registered seed 0 on the grid n = 48, 64, 96 (supplementary 5-seed means: peak 0.514, $\mu(96)$ = 0.085, $\mu(300)$ = 0.108). B2n and B3n: 10 seeds ("sep.": separate batching); B3n's low and high points are 2.85 and 4.65 (sep. 2.65 and 4.35). B4v and B2v: vLLM, variant V1 (simulator token costs scaled to vLLM's all-miss rate), 10 seeds, turns-based rates. B3n is blind in time; B2n, B4v and B2v were pushed after their jobs were submitted, before any run had finished (§5.7); B4n (3-seed mean) is not blind.

	predicted	real
B1	λ_{miss} 0.0722 (turns) 11,220 tokens/s λ_{miss} 0.0694 (sessions)	0.0715 (+0.9%) 11,243 (−0.2%) 0.0717 (−3.1%)
B2	all-stable edge 0.14 (3 seeds), 0.15 (20) at 0.18: 2/3 stable (15/20)	in [0.143, 0.179) 0.179 unstable
B3	PASS at all rates 0.09–0.16 20 seeds: sep. fails at 0.16 token ratio 1.003–1.027 (0.09–0.10) 1.025–1.043 (0.11–0.12) backlog 171–234 (0.09–0.10) 253–313 requests (0.11–0.12) good-arm hit 0.929–0.937 trapped 3/3 at 0.09 and 0.10; 20 seeds: 14/20, 18/20	PASS at 0.0966, 0.1180 (0.16 not run) 1.009–1.025 (0.0966) 1.025–1.029 (0.1180) 194–226 requests 277–325 0.921–0.934 trapped 2/3 at 0.0966
B4	peak n = 64, μ 0.523 (900k) 0.5-hit crossing ≈80 $\mu(96)$ = 0.079, $\mu(300)$ = 0.109	n = 64, 0.479 (+9%) ≈93 (≈91 on B4's grid) 0.123, 0.101
B2n	all-stable edge 6.5; sep. 6.0 at 6.5: 10/10 stable; sep. 8/10 at 1.5–5.5: 10/10 stable, both	in [5.5, 6.5) 6.5 unstable stable
B3n	PASS at 2.85, 4.65; sep. 2.65, 4.35 trapped token ratio 1.086–1.109 (low) 1.119–1.140 (high) trapped backlog 460–643 (low) 1067–1333 requests (high) good-arm hit 0.928–0.936 BURST trapped 9/10 to 10/10	PASS at 2.5, 4.0 1.074–1.078 (2.5) 1.106–1.124 (4.0) 549–594 requests 1089–1130 0.929–0.935 3/3 at both
B4n	peak n = 72, μ 0.532 0.5-hit crossing 94; hit 0.42 at 96 cap 64, 0.35% inside the rule	64–80, 0.479 (+11%) ≈93; 0.42 64
B4v	peak n = 24, μ 0.215; cap 20 hit 0.74 at n = 28, 0.46 at 32 0.5-hit crossing 31.4	n = 20, 0.211 (+2%); 20 0.66, 0.08 29.1
B2v	all-stable edge 2.0; at 2.5: 7/10	in [2.0, 2.5); 2.5 unstable

B3), and good-arm hit fractions of 0.929–0.935 lie inside the predicted 0.928–0.936. B3n had also warned that at its higher point the trapped token rate sits close to the registered tolerance of 15%, so that at higher rates the rule would stop calling a stuck run trapped; the real 1.106–1.124 at 4.0 λ_{ref} is inside the tolerance, by 2.6 to 4.4 percentage points.

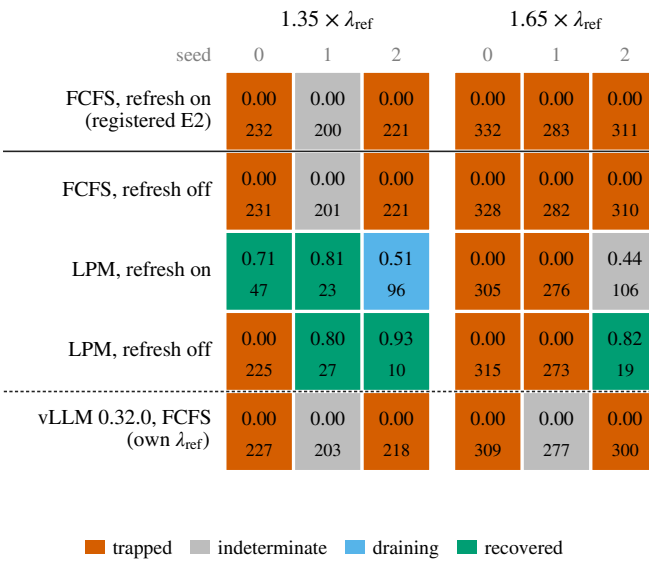


Figure 5: Exploratory policy grid, BURST arm only, 300k pool. Rows: FCFS with the waiting-prefix refresh (the registered runs), FCFS without, LPM with, LPM without (under LPM the refresh acts only during the FCFS fallback above 128 waiting requests); below the dashed line, the exploratory vLLM 0.32.0 runs with FCFS (§5.7), at multiples of its own λ_{ref} (0.0657 sessions/s). Columns: 1.35 and 1.65 λ_{ref} by seed. Colour: registered state (RECOVERED judged on the last 300 s); numbers: hit fraction and mean sessions in the system over the evaluation window.

B1n and B4n are not blind. The native curve and λ_{miss} had been measured and summarised in our project status notes (`notes/STATE.md`) before this round's specification was written, and the specification's finer grid near the cliff (n = 72, 80, 88) may reflect that; only the LLM agent that ran the predictions was instructed not to read those data and, by its own record (not checked independently), did not. For completeness: B1n's λ_{miss}, 0.0720 sessions/s and 11,239 tokens/s, is within 0.4% of the real 0.0718 and 11,282. B4n put the peak at n = 72, which the real grid does not contain, with μ 11% above the real peak at 64–80; it put the 0.5-hit crossing at 94, against about 93, and the cap at 64, as registered. It flagged that cap as knife-edge ($\mu(64)$ only 0.35% above the 5% threshold, where a slightly lower value would have given 72) and the cliff cells n = 80–96 as bistable across seeds; the real cliff lies somewhere between 80 and 96.

5.6 Exploratory policy grid

Three other engine cells ran the BURST arm at the registered rates and seeds (Fig. 5) under exploratory registrations derived

from E2's, with no control arms. In these runs the refresh made no difference for FCFS: all six runs without it, which share their arrivals with the registered runs, have the registered runs' states, hit fraction 0.000 and backlogs within 1.3% of theirs (201–231 sessions at 1.35 λ_{ref}, 282–328 at 1.65), as the source predicts: the refresh moves waiting prefixes up the LRU order but does not pin them. Both rates lie inside the window; near λ_{miss}, with a cap or with the native pool, the refresh was not tested.

LPM partly helps. From the 200-session burst, at 1.35 λ_{ref} it recovered in 4 of 6 runs and was draining in a fifth (hit fractions over the evaluation window; RECOVERED is judged on the last 300 s, where every recovered run was at 0.92–0.94; with the refresh: recovered at 0.71 and 0.81, draining at 0.51; without: recovered at 0.80 and 0.93, trapped once). At 1.65 λ_{ref} it was trapped in 4 of 6, indeterminate in one (hit 0.44) and recovered in one (0.82). The paths behind these window means differ from FCFS's, which had no hits from 100 s after the burst in all twelve FCFS runs (a post-hoc reading in 100 s windows). Each LPM run that ended trapped kept some hits until 1500–1900 s after the burst (3900 s in one run) and then had none; each run that recovered first exceeded a hit fraction of 0.8 3100–6500 s after the burst and stayed above it to the end, so that had the runs ended 2000 s earlier, only two of the five would have met the RECOVERED rule. This is a finite-burst result for the BURST arm alone, not an estimate of LPM's congested service rate μ_∞; with mixed outcomes in every cell, three seeds do not bracket a threshold. In deep congestion stock LPM stops re-sorting its queue, so, on our reading of its source, Proposition 1 applies to it (Appendix E); consistent with this, Stage C's LPM run at $n = 300$ had zero hits with a queue above 128, and round 5 of our earlier simulator, which modelled the 128-request fallback, found no LPM advantage in congestion. In the same earlier, uncalibrated simulator, pure LPM without the fallback has a congested service rate of about 1.17 times that simulator's own λ_{miss} and recovered from 300 cold sessions only up to about 1.1 times it (§7); these are deep-congestion quantities and do not compare directly with recovery from a finite burst. We do not claim that LPM fails, and we have not isolated why it recovered. Post hoc, the timing matches the 128-request fallback: in the five runs that recovered, the engine's waiting queue (5 s samples) never exceeded 128 after the burst (at most 116–126), and in each of the five that ended trapped the last hit fell in the 100 s window in which the queue last stood at or below 128. This is a correlation in ten runs, not an isolation. Other candidates are partial prefix hits and LPM's own prefix matching, which moves the cached part of every waiting request's prefix to the recent end of the LRU order on each scheduling attempt; the simulator omits both. Under LPM the waiting-prefix refresh is skipped, because that matching already touches the waiting prefixes, so the two LPM configurations differ only while more than 128 requests wait and LPM has fallen back to FCFS. Seed 0 at 1.35 λ_{ref} recovered with the refresh and was trapped without it; since the refresh made no difference under FCFS, single runs near LPM's threshold appear sensitive to small differences.

5.7 A second engine: vLLM

vLLM 0.32.0 passed its smoke test on the same GPU type (token ids and cached-token counts returned, flush verified) and ran with the SGLang cell's settings where they exist: FCFS, prefix caching on, chunked prefill of 8192 tokens, at most 256 running requests and a 300,000-token pool (18,750 blocks of 16 tokens). With prefix caching off, its all-miss capacity (vLLM E0a, three seeds) is 0.0657 ± 0.0030 sessions/s turns-based and $10{,}349 \pm 81$ uncached prefill tokens/s, 8% below SGLang's 0.0715 and 11,243 on the same model, GPU and workload.

Six exploratory BURST runs came first, under a registration derived from E2's: the multiples 1.35 and 1.65 of vLLM's own λ_{ref} (0.0886 and 0.1083 sessions/s), seeds 0, 1, 2, and E2's protocol and trap rule with vLLM's reference rates. Their window and multiples were inherited from the SGLang cell, not measured: these runs preceded vLLM's own closed curve and from-empty edge, and no NOBURST or CAPPED arm ran with them. Before the burst all six runs were in the good state (hit fraction 0.93–0.94 from 300 s to the burst; a post-hoc reading). After it, all six were at zero hits from the 100 s window starting 100 s after the burst to the end of the run (also a post-hoc reading), with an uncached prefill rate of 0.997–1.031 times vLLM's all-miss rate and mean backlogs of 203–227 sessions at 1.35 and 277–309 at 1.65 λ_{ref} (Fig. 5, bottom row). The registered rule calls four of them trapped. The other two are indeterminate by the slope test: one has a flat backlog near 203 sessions (slope 0.0002 against a threshold of 0.0115; 1.35 λ_{ref}, seed 1), and in the other the backlog grew from 269 to 294 sessions over the window, at 0.0175 against 0.0213 (1.65 λ_{ref}, seed 1). Both are seed 1, as are the indeterminate SGLang FCFS runs at 1.35 λ_{ref} with and without the refresh. A vLLM run's arrivals are a subset of the SGLang run's at the same seed and multiple, and seed 1 delivered 5.5–7.9% fewer arrivals than nominal in these windows, so the labels on the two engines agree on the same workloads rather than on independent ones. So the same all-miss state appears on a second engine after the same burst. Without control arms these runs are not a registered paired test: they do not show that vLLM's good state would have lasted the run without the burst, or that a cap would have recovered it.

vLLM's own closed curve (Fig. 2, right; one run per n) has the shape of SGLang's at the same pool. $\mu(n)$ peaks at 0.219 sessions/s at $n = 20$, with $\mu(24)$ 0.5% lower, so the peak is a tie at one run per n; $\mu^*/\lambda_{\text{ref}} = 3.34$ (3.21 turns-based). The hit fraction is 0.93 at $n \le 20$, 0.88 at 24, 0.66 at 28, 0.08 at 32 and 0.0075 at 40; it crosses 0.5 near $n = 29.1$, close to N_{lb}, and μ falls 80% from $n = 24$ to 32. The closed curve had no cache hits from $n = 48$ on, as with SGLang's FCFS; Proposition 1, a limit statement, is consistent with this but does not predict it. Like SGLang's, the curve is not monotone beyond its peak: in completed sessions it falls to 0.63–0.67 λ_{ref} at $n = 32$–48 (0.83–0.90 turns-based), whose windows held old, long turns (served prompts 12–17% above the stationary mean), and rises to 1.60 λ_{ref} at $n = 300$ (1.59 turns-based). vLLM preempted running requests in every closed run from $n = 24$

on (12 to 72 per run from $n = 32$ on), which the proposition allows on our reading of vLLM's source (Appendix E).

From an empty cache and queue (Fig. 3, bottom; one 3000 s run per rate) vLLM stayed stable at 1.25, 1.5, 1.75 and 2.0 λ_{ref} (0.082–0.131 sessions/s), ending each run with an empty queue and with 92–93% of turns hitting, and was unstable at 2.5 λ_{ref} (final queue 82, rising by 94 over the last 1500 s) and 3.0 (247 and 175). The edge lies between 2.0 and 2.5 λ_{ref}, as on SGLang with the same pool, so vLLM's window is $(1, 2.0)\,\lambda_{\text{ref}}$. These runs also used seed 0 and received 8.6–13.3% more arrivals than nominal (279 to 642 sessions; the runs labelled 2.0 and 2.5 λ_{ref} received 2.21 and 2.74 λ_{ref}), but the unstable 2.5 run's realized 0.180 sessions/s is below the measured peak (0.219, or 0.211 turns-based), so here, as at 300k on SGLang and unlike with the native pool, the edge lies below the peak.

Blind predictions for this cell, B4v for the curve and B2v for the edge, were made with the calibrated simulator in two variants, unchanged and (V1) with its token costs scaled so that its all-miss token rate matches vLLM's measured one (a fit to vLLM's E0a, not a prediction), each in both batching modes, and are frozen in `notes/PREDICTIONS_vllm.md`. They were committed and pushed to GitHub before any of the curve and edge runs had finished, but 22 minutes after the jobs were submitted, when two of them had been running for almost as long; they are blind in that no result existed, not in submission order. The peak level and the cap held: B4v (V1, mixed batching, 10 seeds) put the peak at $n = 24$ with 0.215 sessions/s turns-based, against the real 0.211 at $n = 20$ (0.210 at 24), and the cap at 20. The real cliff is steeper and earlier than predicted: hit fractions of 0.66 against 0.74 at $n = 28$ and 0.08 against 0.46 at $n = 32$ (0.46–0.53 over the four simulator configurations), and a 0.5 crossing near 29.1 against 31.4–32.6. B2v put the edge at 2.0 λ_{ref} in all four configurations (V1 with mixed batching: 10 of 10 runs stable through 2.0, 7 of 10 at 2.5 and 3 of 10 at 3.0), as measured. The simulator does not model vLLM's block-level eviction or its scheduler and token budget; like vLLM, and unlike SGLang, it has no waiting-prefix refresh.

A registration for vLLM was then filled by the formulas used for SGLang (Appendix B) from this curve, vLLM's E0a and five of the six E0b runs (the 3.0 λ_{ref} run was still running; it came out unstable, which leaves the bracket unchanged). It gives the window $(1, 2.0)\,\lambda_{\text{ref}}$ with $\lambda_{\text{ref}} = 0.06566$ sessions/s, the points 1.35 and 1.65 λ_{ref} by the one-third/two-thirds rule, which was written in our status notes before any vLLM curve or edge existed, and $n_{\text{cap}} = 20$, flagged ambiguous because $\mu(24)$ lies within 0.5% of the peak. The protocol, the burst and the trap rule are E2's. The points equal the multiples of the exploratory runs; the rule that selected them was adopted for vLLM (`836063a`) after their outcome was known, and the BURST arm reuses their seeds 0–2 and so their arrivals: it is not an independent replication of them. The registration was committed 37 s before the 18 jobs of the paired test were submitted, but reached GitHub 2 min after the submission and 53 s after the first job had started, before any job had finished; by our definition (§4.3) this test is therefore registered but not pre-registered. The third prediction round, frozen before the registration existed, also predicted this test (B3v): a pass at both points in all four simulator configurations, with every simulated BURST run at zero hits (7 to 9 of 10 per point trapped by the rule, the rest indeterminate), NOBURST good in 10 of 10 and CAPPED at a cap of 20 recovered in 10 of 10. Because the session that specified that round knew the exploratory BURST outcomes, the BURST-arm part of B3v is blind at most on the predicting agent's side, and there only by instruction and by the agent's own unchecked record: its copy of the repository held those runs (Appendix C).

[pending: verdict of the registered vLLM paired test (18 jobs 17166121–17166138; BURST, NOBURST and CAPPED at 1.35 and 1.65 λ_{ref}, cap 20, seeds 0, 1, 2), from the registered analysis at the registration commit: states per arm and seed, hit fractions, uncached prefill rates, backlogs and slopes, realized arrival rates, gate waits and turn latencies, reported as for E2, and compared with B3v; state the verdict as the registered analysis's pass or fail, not with its pre-registered and decisive labels, which it assigns by commit order (§4.3).]

6 Robustness in the simulator

The real test fixes one workload, one burst size and one horizon; we varied each in the calibrated simulator (mixed batching). Each workload gets its own window $[1, hi]$ in multiples of its own λ_{miss} (hi: the largest multiple at which all 10 from-empty runs stay stable) and is tested one third and two thirds of the way across it; a point passes if each arm meets its criterion in at least 7 of 10 seeds, counted separately per arm (the registered real-engine rule instead counts seeds in which all three arms succeed). The repository holds the per-run summaries of these simulations, from which every number below is computed, but not the 100 s window series behind them.

Tool-time laws. With exponential tool times of mean 1, 2 and 5 s, a lognormal law (mean 5 s, coefficient of variation 2) and a bimodal law (80% at 1 s, 20% at 20 s), λ_{miss} stays within 0.0718–0.0730 sessions/s: with the token profile fixed, it does not depend on the tool-time law. Shorter tool times raise the peak ($\mu^*/\lambda_{\text{miss}}$ is 5.25, 4.57 and 2.97 for means of 1, 2 and 5 s) and widen the window ($hi = 3.0$, 2.5, 2.0); a heavy-tailed or a bimodal law of about the same mean (5 s and 4.8 s) leaves the edge at 2.0. The test passes at all 10 points (Fig. 6): NOBURST and CAPPED succeed in 10 of 10 seeds everywhere, and BURST is trapped in 7 to 10 of 10.

Tool times from real agent traces. We replaced the tool-time law by empirical tables from SyFI TraceLab, real sessions of two commercial coding agents from 52 pseudonymous users (release v0.0.2, a later snapshot than the one the paper describes) [78], and from mini-SWE-agent v2.0.0 [64, 71] on the Verified subset of SWE-bench [14, 36]. From TraceLab we take the per-turn tool time (mean 8.51 s, median 0.237 s, coefficient of variation 4.65) and, as an upper bound on it, the idle gap between requests (mean 14.52 s). Both are capped

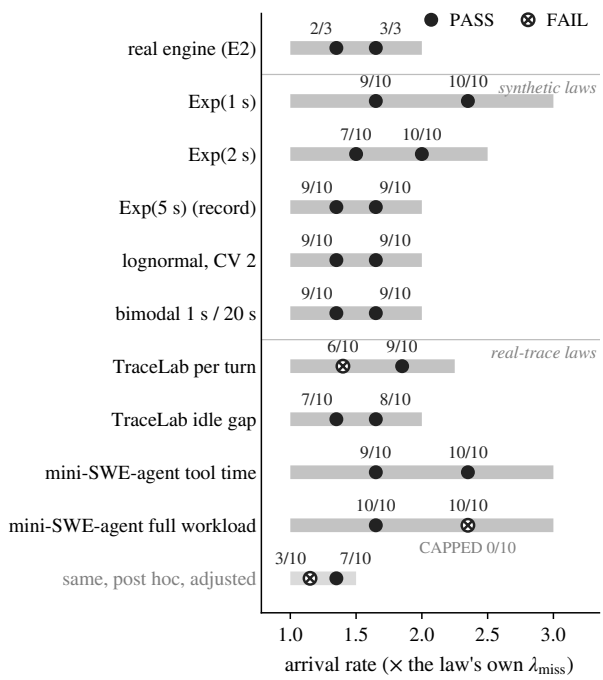


Figure 6: The paired test across tool-time laws (simulator). Gray bar: from-empty window $[1, hi]$ in multiples of each law's λ_{miss}. Markers: test points, filled = pass, crossed = fail (each arm counted separately, in ≥ 7 of 10 seeds); number = BURST runs trapped of 10. Top row: the real engine (3 seeds; registered rule, all three arms in at least 2 of the same seeds). Bottom row: labelled post-hoc rerun of the mini-SWE-agent full workload. The two mini-SWE-agent full-workload rows use different units: the declared row multiples of its declared λ_{miss} (0.0120 sessions/s), the post-hoc row multiples of the long-window λ_{miss} (0.0141 sessions/s), in which the declared 1.65, 2.35 and 3.0 are 1.40, 1.99 and 2.55.

at 600 s, which affects 0.18% of turns and 0.38% of gaps but removes 38% and 66% of their raw means (13.6 s and 42.9 s); the per-turn series includes sub-agent runs and can include waits at permission prompts. Only TraceLab's tool times are used: its prompts average 172k tokens, more than the testbed's 49,152-token context limit. The mini-SWE-agent law comes from 500 published bash-only trajectories, the same 100 instances for each of five models (mean 1.40 s, median 0.104 s, coefficient of variation 4.29); 0.56% of its steps hit the harness's 60 s command limit, so the law is specific to that limit. λ_{miss} moved by −0.8% to +1.3%. The peak moved from 20 in the exponential control to 32 (TraceLab per turn), 40 (idle gap) and 24 (mini-SWE-agent), and the cap to 32, 40 and 20; at every point of these three laws the CAPPED arm recovered in 10 of 10 seeds. The exponential control reproduced the record workload exactly and passed at both points. Of the six points of the three laws, five passed; the failure, TraceLab per turn at 1.40 λ_{miss}, is the slope rule: 6 of 10 BURST runs met it, all 10 were stuck below a hit fraction of 0.2, and on 20 fresh seeds (post hoc) 17 met it. A secondary workload with mini-SWE-agent's mean turn and token counts (geometric turn counts) passed at 1.65 λ_{miss} and failed at 2.35. There CAPPED (cap 12, the curve's peak) had left the trap, but at $\mu(12)-\lambda \approx 0.022$ sessions/s it needs about 9000 s to drain 200 sessions, longer than the 7500 s the run lasts after the burst; NOBURST was good in 8 of 10, and the other two collapsed without a burst. This workload's sessions outlive the declared measurement windows (λ_{miss} is 0.0120 sessions/s as declared, 0.0141 with a 20 times longer window, in which units the failed point is 1.99); a labelled post-hoc rerun with longer horizons gives, in those units, a narrower window, [1, 1.5], a pass at 1.35, and at 1.15 a failure of the slope rule (3 of 10 trapped, all 10 stuck below a hit fraction of 0.2).

Burst size and horizon. At 1.35 and 1.65 λ_{miss}, a burst of 25 sessions never traps (0 of 10), 50 is a coin flip (5 and 6 of 10 trapped), and from 100 up every run is stuck below a hit fraction of 0.2. In runs extended to 30,000 s after the burst, no trapped BURST run ever returned to a 100 s window above a hit fraction of 0.5, all 20 ended with 468 to 1065 waiting requests, and no NOBURST run ever exceeded 50.

The slope rule is the fragile part. Over the 100 BURST runs of the tool-time test the registered rule labels 90 trapped and 10 indeterminate, while all 100 are stuck at zero hits with an all-miss token rate. At 1.35 λ_{miss} with 20 fresh seeds, 3 are indeterminate (Wilson interval [0.05, 0.36]; median slope $0.77(\lambda-\lambda_{\text{miss}})$ against a threshold of 0.5), so the real engine's one in three there is not surprising. The threshold presumes that a trapped engine completes sessions near λ_{miss}, but the registered SGLang BURST runs at 300k completed 1.13–1.35 λ_{ref} in the evaluation window (§3.3); the indeterminate one, seed 1, received 1.27 λ_{ref} there instead of the nominal 1.35 and completed 1.28, so its backlog stayed flat. Over 31,200 s runs the rule labels only 10 to 12 of 20 stuck BURST runs trapped, depending on the window. Post hoc, a rule on the hit fraction alone would have called all six real E2 BURST runs trapped, and all six exploratory vLLM runs, of which the registered rule calls four trapped (§5.7); the vLLM runs reuse the SGLang runs' seeds, and both of their indeterminate runs are again seed 1, so the two engines' labels are not independent. We report the registered rule. Its token criterion has the opposite weakness at high multiples of λ_{miss}, where a trapped engine prefills more than the all-miss token rate: 1.106–1.124 times it at 4.0 λ_{ref} with the native pool, against a tolerance of 1.15 (§5.4).

The cap must follow the measured curve. The right cap, 20 for the workload of record at 300k, moves to 32 or 40 with the TraceLab tool-time laws (it stays at 20 with mini-SWE-agent's) and to 64 with the native pool, where it recovered in 6 of 6 registered runs. With mini-SWE-agent's full workload the peak falls to 12, and a cap of 20 would sit past the peak; a cap c above the new peak leaves $(\mu(c), \mu^*)$ open in the quasi-static model (§3.4).

7 Published mechanisms in the earlier simulator

In the fluid model of §3.3 the recovery threshold from deep congestion is at most μ_∞, so a mechanism can raise it only as far as it raises μ_∞. We compare mechanisms by $\mu(300)/\lambda_{\text{miss}}$ as a proxy for μ_∞ (Fig. 7) in the earlier, uncalibrated simulator. Each rule was compared, by reading the code and not running it, with the released code where it exists (PEEK; the Continuum preview, which omits the paper's estimator) or with the paper (CONCUR, for which we found no code); where a difference was small and unambiguous we added a projection of the released rule beside the historical row, and the differences that remain are stated below. That simulator's assumed costs are about 1.5 times the fitted ones (λ_{miss} 0.0474 against 0.0722 sessions/s), and its curve peaks at 4.07 times its own λ_{miss}, against about 3.0 in the calibrated simulator and 3.26 λ_{ref} on the engine. Its absolute rates therefore do not carry over. As in Stage C, all sessions start at $t = 0$, but each run is measured over 25,000 s after a 5000 s warmup (static caps: 30,000 s), about four mean session lifetimes; FCFS reads 1.04 λ_{miss} over the first half of the window and 1.01 over the second, so the synchronized-start bias of §8 is a few percent here. These are results about the rules in a workload with private prefixes and exponential 5 s tools, not benchmarks of the named systems.

Orderings. FCFS gives 1.024 ± 0.020 with hit fraction 0.000 and the GPU always busy; with the 1.5% standard error of λ_{miss} included the interval is 1.02 ± 0.04, consistent with Proposition 1, which covers this simulator's request-level FCFS. Pure LPM (without SGLang's 128-request fallback) gives 1.174 ± 0.023 and recovers from 300 cold sessions over 150,000 s up to 1.1 λ_{miss}, not at 1.25. PEEK [70] protects prefixes needed by queued requests and adds a cache-aware scheduler. Our earlier rows implemented only the protection under FCFS order. As a soft ranking, which is how PEEK's eviction hook protects (finite priorities), it gives the same throughput as FCFS seed for seed: queued requests' KV is protected but still evicted before FCFS admits them (2,831–3,128 such evictions per run), so no request is admitted with its KV, consistent with Proposition 1. A hard pin, a counterfactual PEEK does not apply, gives 0.919. The released scheduler serves warm requests first; projected onto single-session prefixes with its eviction score, it gives 1.176 ± 0.021, equal to pure LPM within 0.001 ± 0.014, and recovers from none of 5 bursts at 2 λ_{miss}. Here PEEK is an LPM-class ordering, not FCFS; we did not evaluate it on shared-prefix traffic, for which it is designed.

TTL pins. Continuum keeps a program's KV after a non-final turn for a time-to-live chosen by a cost model, which may be zero [41]. Its released preview omits that estimator: it pins for 2 s, and only when the parsed tool's historical mean is at most 2 s, so it would rarely pin our 5 s-mean tool calls. We did not implement the paper's estimator. Our rows are therefore fixed, unconditional TTLs of $\tau = 5$, 10 and 20 s on every non-final turn, with Continuum's waiting order (pinned programs first). Under the preview's release rule (when an allocation fails and nothing runs, release the pin that expires last) they give 2.584 ± 0.035, 3.505 ± 0.064 and 4.376 ± 0.048; at $\tau = 20$ s the system recovers from a cold start up to 4.0 λ_{miss} and from 5 of 5 bursts. Under the rule we read in the paper's deadlock-prevention text (when a request does not fit, unpin the latest-arriving programs until it fits, also while work runs), $\tau = 10$ and 20 s give 1.16–1.17 and recover from no cold start at 1.1 λ_{miss} or above and from no burst. Our earlier rule, which waited for expiry while the GPU idled, gave 1.719 at $\tau = 20$ s. With plain FCFS order instead of Continuum's, under that earlier rule, the pins gave 0.29, 0.17 and 0.10 at $\tau = 5$, 10 and 20 s (3 seeds), with the GPU busy 33%, 17% and 8% of the time: that simulator keeps an expired pin while the session's next request waits, which under FCFS lasts the rest of the queue wait. This is consistent with Proposition 1. These are results about configured pins, not about Continuum.

Adaptive caps. CONCUR adds 2 to a window on active agents when KV usage is below 0.2 and halves it when usage exceeds 0.5 and the hit rate is below 0.2 [12]; we found no released code. Our session-gate surrogate of this equation gives 3.009 ± 0.081 and recovers from 5 of 5 bursts. It depends on the usage signal: if finished sessions' KV, still cached and evictable, counts as usage, 12 of 18 cells stall at our surrogate's minimum window of 4 at 0.915–0.935 λ_{miss}; of five exploratory sensitivities, one (a 5 s control interval instead of our surrogate's 10 s) raised the rate by 0.184 ± 0.167, an unadjusted pointwise interval. Pause and resume of admitted agents is not modelled. So 3.0 is the surrogate's rate under a live-sessions-only usage signal, not deployed CONCUR's threshold.

Static caps. By Proposition 2 a cap c holds a large population at $\mu(c)$: 4.04 ± 0.25 at $c = 20$, 2.39 ± 0.35 at $c = 28 \approx N_{\text{lb}}$ and 1.08 ± 0.17 at $c = 36$ (3 seeds), against a peak of 4.07 at $n = 22$. The cliff makes the choice fragile: a cap about a quarter above N_{lb} is back near λ_{miss}.

Admission by memory demand. ThunderAgent periodically compares the programs' KV demand with the capacity, counting a program in a tool call at a weight that decays with its tool time (2^{-t} in its experiments). When demand exceeds capacity it pauses whole programs, those in tool calls and those with the shortest contexts first, and it restores them when demand falls; both watermarks are set to 1 in its evaluation [37]. NVIDIA Dynamo ships a port that pauses programs at tool boundaries, by default when utilization reaches 0.95 of the pool, down to 0.80, and resumes them at 0.85 or below [20]. This is an implemented form of a session bound, sized by memory demand rather than count; we have not evaluated it. Proposition 2 concerns a FIFO count cap whose sessions keep their slots, so it does not predict a gate that bounds current demand and pauses admitted programs. The static-cap rates

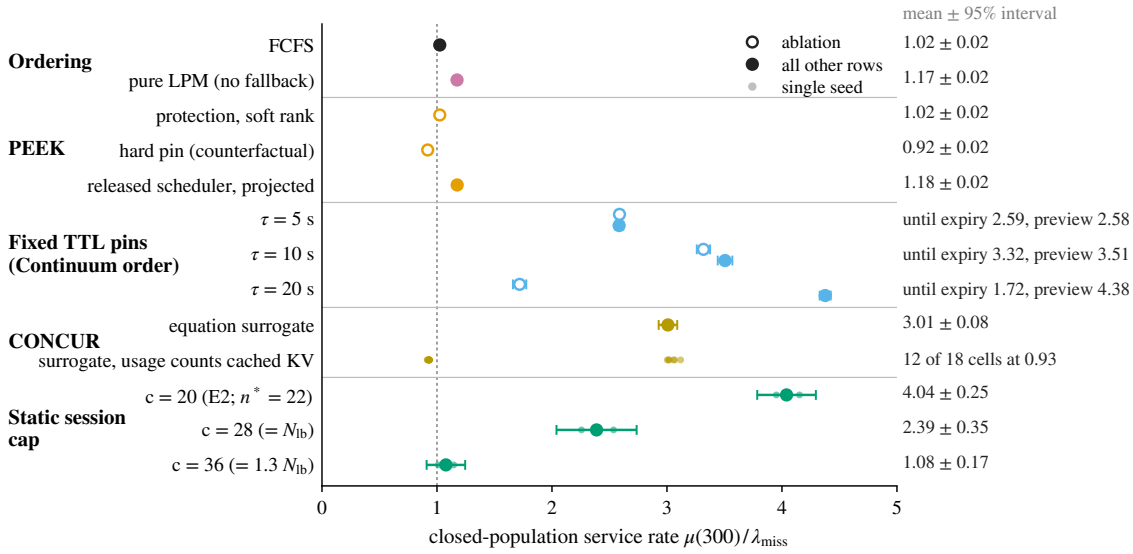


Figure 7: $\mu(300)/\lambda_{\text{miss}}$, our proxy for the congested service rate μ_∞, per mechanism in the earlier, uncalibrated simulator: mean over 10 seeds with 95% t intervals, in units of that simulator's λ_{miss} (0.0474 sessions/s; not comparable with Figs. 2–3); the intervals omit the 1.5% standard error of λ_{miss}. Hollow: ablations (PEEK's queued-prefix protection under FCFS order, as a soft rank and as a hard-pin counterfactual PEEK does not apply; fixed pins held until expiry even with the GPU idle). Filled: all other rows; for PEEK, a projection of its released scheduler; for the pins, fixed TTLs under the preview's release rule (values: held until expiry, preview). LPM is pure LPM, without SGLang's 128-request fallback. CONCUR: a surrogate of its control equation; small dots: 18 cells in which finished sessions' KV counts as usage. Static caps $c = 20$ (E2's cap; this simulator's n^* is 22), 28, 36: $\mu(c)$ of the closed FCFS curve (3 other seeds, not paired with the other rows).

above are not a prediction for either system.

Why the ranking comes out this way. Under request-level FCFS, with or without protection that keeps FCFS order, μ_∞ cannot exceed λ_{miss} when (A3) holds (Proposition 1); we conjecture (Remark in §3.4) that pure resident-first orderings raise it through tool calls shorter than the congested lifetime of idle KV, and for pure LPM the simulator agrees. A static cap c holds the engine at $\mu(c)$ (Proposition 2), so at most at the peak, 4.07 at $n = 22$ (3 seeds). Pins at $\tau = 20$ s gave 4.38, above every static cap we ran and above the peak, though within the peak's 3-seed interval (±0.36). In this simulator they also serve pinned programs first and keep a returning session's KV for at least τ, and the set of sessions they bound depends on memory, not on a count; we have not separated these effects. On the real engine, stock LPM recovered from a 200-session burst at 1.35 λ_{ref} in 4 of 6 runs (§5.6); that finite-burst result does not compare directly with these deep-congestion rates, and in deep congestion stock LPM stops re-sorting its queue, so, on our reading of its source, Proposition 1 bounds its μ_∞ by λ_{miss} when (A3) holds.

8 A diagnostic, and what it implies

Choosing the number of admitted jobs from the throughput curve, by paging heuristics [19] or from measured throughput [28, 61], is the classical selection of a multiprogramming level. Stack-distance estimators size a KV tier to the working set, for a target hit rate (KVSET [42]) or for the host offload tier (EfficientAgent [62]), and Yao et al.'s mean-field estimator sizes memory for a target hit ratio when turn gaps are exogenous [72]. What the window adds is that the from-empty edge, a finite-horizon stability limit in the sense of Kleinrock and Lam's first exit time [39], and not only μ^*, bounds the operating range; that a cap protects against history, not only throughput; and that the right cap moves with the tool-time law, the token profile and the pool.

The window of Eq. (2) comes from three measurements that an operator can make before deployment, on the engine, model, pool and workload that will be served. (1) *All-miss capacity*: run a saturated closed population with prefix caching disabled and measure the uncached prefill token rate, which is stable across seeds; check saturation by measuring at two populations (for example 32 and 64 sessions) and use the larger rate, since a population that is not saturated understates λ_{miss} (our 32-session reference fell about 2–3% short, §3.4); convert it to sessions by the mean work per session, or use turns per second over $E[T]$ measured over a window longer than a session's lifetime at that population (mini-SWE-agent's sessions outlived the declared 1800 s window, and the turn rate there gave a λ_{miss} 15% low); completed sessions over a short window are noisy. By Proposition 1 and the fluid model of §3.3, under request-level FCFS this is an upper bound on the

recovery threshold from deep congestion if the congested engine computes prompt tokens no faster than the cache-disabled one, and it equals that threshold if, in addition, $\mu(n) \geq \lambda_{\text{miss}}$ at every population beyond the peak and not only in the limit (with zero hits, if the engine computes prompt tokens as fast as the cache-disabled one at every congested population); the heuristic work balance of §3.3 leads to the same conditions for the open system. With zero hits our engine computed prompt tokens at 1.03 times the all-miss rate near the stationary turn mix and at 0.99–1.12 times it over the windows we measured, so (A3) and (A4) hold on it only to within a few percent at the stationary mix and to within 12% over those windows. If sessions share a prefix (system prompt, tool definitions), measure instead with caching on and each turn's private context fresh, so that only the shared prefix can hit, the reference against which we expect Proposition 1 to hold for the private suffixes (§3.4); disabling caching entirely understates the congested rate. (2) *Closed-population curve*: measure $\mu(n)$ on a grid that crosses the cliff, and do not read μ_∞ off the largest n of a short run with a synchronized start; here $\mu(300)$ was 0.108 sessions/s, 51% above λ_{ref}. (3) *From-empty edge*: run open arrivals from an empty engine at increasing rates over the horizon of interest, at least that of the burst test.

If $\mu^* > \lambda_{\text{miss}}$, then in the fluid model of §3.3, under request-level FCFS and the assumptions of Proposition 1, every operating point in $(\lambda_{\text{miss}}, \min(\mu^*, \lambda_{\text{e}}))$ is one sufficiently large burst away from the congested state unless admission is bounded; a steady-state load test there passes without testing recovery. On the engine we tested two rates per pool and one burst size: 200 sessions, about 20 times the 8–12 present at 300k and 6 to 13 times the 16–34 present with the native pool, left all 12 registered SGLang BURST runs at zero hits (11 trapped by the rule); in the simulator 25 never trapped and 50 trapped about half the time. Without a cap, Proposition 1 and the fluid model predict that recovering from deep congestion requires cutting the arrival rate below λ_{miss}, which was 31% of peak throughput at 300k tokens and 15% with the native pool; the earlier, uncalibrated simulator agrees (over 150,000 s, FCFS recovered from congestion at 0.9 times its own λ_{miss} and not at 1.0), but no real run lowered the rate after a trap. Stock LPM, which on our reading of its source stops re-sorting its queue when more than 128 requests wait and so falls under Proposition 1 in deep congestion, recovered from the same finite burst at 1.35 λ_{ref} in 4 of 6 grid runs, and at 1.65 λ_{ref} in 1 of 6 (§5.6); the proposition is a limit statement and does not predict recovery from a finite burst for any ordering. Proposition 2 and the quasi-static model suggest a session cap at or just below the peak, since a cap past the peak leaves $(\mu(c), \mu^*)$ open (§3.4); we take the smallest n within a few percent of the peak (20 at 300k, 64 with the native pool), because the curve has one run per n, and check it with a burst test. We recommend a cap because it needs no engine change, its effect in the quasi-static model follows from Proposition 2 and Eq. (1), and it is the one mechanism we tested on the engine (all 12 registered SGLang CAPPED runs recovered), not because it dominates: in the earlier simulator, 20 s TTL pins with a pinned-first order reached a higher $\mu(300)$ than any static cap (§7). A cap does not make the burst free: it moves the burst's cost to the client as admission delay (per-run maxima of 853–985 s at 300k and 247–361 s with the native pool), instead of turns that take several minutes for the rest of the run (§5.3, §5.4). The delay ends only if the gate drains, which in the quasi-static model needs $\lambda < \mu(c)$ for a cap c (Proposition 2 and Eq. (1)). Whether the gate should queue or reject sessions is a deployment choice we did not evaluate.

The cap must be measured again when the workload or configuration changes. In the simulator the peak moved from 20 to 32 or 40 sessions with the TraceLab tool-time laws and to 24 with mini-SWE-agent's (where the cap stayed at 20), and fell to 12 with mini-SWE-agent's mean turn and token counts. On the real engine it moved to 64–80 with the 955,757-token pool, where the registered cap of 64 drained the burst in every run, and with vLLM 0.32.0 on the 300k pool it stayed at 20–24. A cap sized for one setting can fail in another in either direction. Too small, it holds the engine below the offered load: a cap of 20 under the TraceLab idle-gap law gives $\mu(20) = 1.26\,\lambda_{\text{miss}}$, below both test rates (1.35 and 1.65 λ_{miss}), so the gate would not drain. Too large, it sits on the collapsed side of the curve: with mini-SWE-agent's full workload $\mu(20)$ is 1.45 times that workload's declared λ_{miss}, again below both test rates (1.65 and 2.35). Neither mis-sized cap was run. The procedure sizes a static cap offline and gives no online detector of a trap in progress. Here its three stages took 14–16 A100-hours per pool and engine, and a three-arm burst check about 7.5 more per rate and seed.

9 Related work

Thrashing, caching and metastability. The thrashing collapse and the working-set remedy are Denning's [17, 18], and the decomposition analysis of thrashing instability and the choice of a multiprogramming level are classical (§2); LRU's failure on loops is classical [13, 63], as is its weakness on weak-locality patterns [34]. Metastable failures were named by Bronson et al. [9], studied in production incidents and modelled by Huang et al. [31], analysed with queueing models, simulation and emulation by Isaacs et al. [32] and with continuous-time Markov chains by Alvaro et al. [2], predicted with a queueing model for replicated storage by Habibi et al. [27], and attributed to destabilizing feedback cycles by Farahbakhsh et al. [22, 23]. Gibbens, Hunt and Kelly's networks with random alternative routing are a classical case in which a state-dependent priority rule, trunk reservation, removes a congested mode and its hysteresis [26]; their modes are both stable, whereas our congested regime is transient, and the closer analogue of our cap is multiprogramming-level load control (§2).

Queueing theory of LLM serving. Ao et al. give fluid-guided scheduling with threshold admission [3] and show that in a decode-only model, which abstracts away prefix reuse, the eviction-free fixed point can be unstable, with rate-limited admission and length mixing as remedies; its benchmark GPU

experiments disable prefix caching [4]. In its open-arrival simulations, every load tested between the worst-cycle throughput, the analogue of our λ_{miss}, and the eviction-free capacity accumulated backlog. On our engine, backlog accumulated from an empty start only above the from-empty edge; inside our window the good state persisted from empty for the length of each run, and a burst moved the system out of it. Dai et al. prove throughput optimality of work-conserving policies without cache state, and give a cyclic-routing example where work-conserving scheduling fails [16]; Nie et al. give stability conditions under KV memory constraints for independent one-shot requests whose KV cache is freed at completion [52]; Dong and Cao study flow-controlled admission [21]. Yao et al. derive a mean-field LRU hit ratio for multi-turn conversations with exogenous gaps between turns [72]; the feedback from load to gap length to hit rate is the loop studied here.

Prefix and KV mechanisms for agents. PEEK protects prefixes needed by queued requests and adds a cache-aware scheduler [70]; its "scheduling-induced thrashing" is LPM breaking ties by arrival order, not the 128-request fallback. Continuum keeps a program's KV across a tool call for a time-to-live chosen by a cost model, which may be zero, and reports mean tool times of 925 ms on SWE-Bench (standard deviation 3550 ms) and 1923 ms on BFCL [41, 57]. CONCUR controls the number of active agents with AIMD and reports SGLang's hit rate falling from 77.7% at batch 32 to 35.4% at 40 [12]; MARS sets an admission window by AIMD from GPU KV pressure and CPU tool pressure, and reports that under heavier load its baselines' SLO-aware goodput collapses to near zero while their token throughput stays high [67]. On vLLM, Yuan et al. report the prefix-cache hit rate falling from 98.85% to 16.78% as the maximum number of concurrent Terminal-Bench agents goes from 15 to 20 [75], and VAMP calls the sharp rise in time to first token once reusable prefixes outgrow the KV pool the prefix-cache cliff, and measures stock vLLM at a 0.19% prefix hit rate on a recorded SWE-bench agent replay [38]. ThunderAgent pauses whole programs when their KV demand, with programs in tool calls counted at a weight that decays with tool time, exceeds the capacity, and reports that baseline systems collapse in throughput as the number of parallel workflows grows, a closed population [37]. CacheWise evaluates coding agents with sessions started together [65]; XPerf, a benchmark, sees hit rates fall toward saturation in fixed-rate runs at increasing rates [66]. Other systems manage idle KV across tool calls or workflow steps by offloading, prefetching or retention [1, 7, 8, 46, 53, 69, 76], schedule agent programs [44, 48], cache conversation state hierarchically [25, 58, 74] (Mooncake's scheduler also rejects requests early under overload), admit, place or reserve capacity for agent sessions by tool readiness, KV residency or likely returns [35, 43, 50], or make locality-aware scheduling fair [11]. Eviction-order work [47, 54, 55, 68] changes which idle state is evicted; RR-Evict assumes that the active trajectories fit in the cache, ScaleSim prefetches and evicts each agent's state by the estimated order of the agents' next requests, and Liu et al. replay traces without feedback from queueing to arrivals, as do Arya et al. for production Microsoft M365 agent traces, where LRU with a pooled budget of a tenth of the busiest hour's no-eviction demand makes 63% of Deep Research calls recompute [5]. MISA-T, for RL rollouts under a concurrency ceiling, reports vLLM Router's prefix-cache hit rate falling from 92.4% to 4.5% at a high ceiling, describes the positive feedback behind it, notes that at prefill saturation the request rate is bounded by prefill throughput over the mean uncached tokens per request, and interrupts the loop with an adaptive admission cap for new sessions derived from observed KV demand and overload pressure [30]; it has no open arrivals, bursts or recovery test. Other close neighbours: Metronome calls a collapse in real-time speech-model serving metastable, in the sense that identical runs collapse or survive near a threshold, but there each session's KV stays pinned for the whole conversation until the pinned KV exhausts the pool, a different mechanism from eviction and recomputation of reusable prefixes [49]; and, known to us only from their abstracts, AgentServeSim simulates agent serving with KV retained across tool gaps [60] and ServeTwin simulates multi-turn agent workloads while tracking KV-cache evolution [56].

Position. Hit-rate collapse with too many active agents is published [12, 30, 37, 66, 75], as are thrashing from repeated eviction and recomputation under concurrent sessions [8, 65], thrashing caused by scheduling order [70] and the feedback that sustains it, between eviction, cold prefill and queueing under a concurrency ceiling [30] and as a self-reinforcing eviction–recompute cascade under memory pressure [38]; session-aware admission is implemented or proposed [20, 30, 35, 37, 43, 67]; we claim none of these. To our knowledge no prior work on LLM serving shows the open-system, history-dependent version (a good state that persists from empty, a burst that leaves the system congested, and a cap that prevents it) against a no-burst control with a pre-registered rule on a real engine; in general distributed systems, the cache-loss version is Bronson et al.'s look-aside-cache case [9], whose persistence Farahbakhsh et al. attribute partly to the database serving its queue in arrival order [23]. Covering our setting would take multi-turn sessions and prefix reuse in the model of Ao et al. [4], and in that of Yao et al. [72] a finite server whose recompute cost feeds back into the gaps between turns; neither paper has these.

10 Limitations and threats to validity

One model and GPU, two pools, two engines. All registered SGLang evidence is for SGLang 0.5.22, Qwen2.5-7B-Instruct and one A100, at two pool sizes: the 300k-token pool limit and the 955,757-token pool SGLang sizes itself. The second answers the concern that the 300k-token pool limit created the cliff: with the native pool the cliff moved out and the registered test passed there too. vLLM 0.32.0 ran only with the 300k-token pool, on the same model and GPU. Its own closed curve and from-empty edge select the same window, $(1,\ 2.0)\ \lambda_{\text{ref}}$, the same test points, 1.35 and 1.65 λ_{ref}, and

the same cap of 20 as SGLang's 300k cell (§5.7). The exploratory vLLM BURST runs preceded those measurements and had no NOBURST or CAPPED arm, so on their own they cannot exclude that a vLLM NOBURST run would also have congested at these rates within 8700 s, or that a cap would not have recovered it. The registered vLLM paired test (§5.7) uses the same seeds, reference rate, multiples and burst, so its BURST arm repeats the exploratory runs' workloads and does not replicate them independently. Larger models, parallelism, other GPUs and offloading are untested.

Synthetic sessions and clients. Sessions are synthetic (random tokens, fixed tool outputs and output lengths, geometric turn counts, exponential 5 s tools) and share no prefixes, which removes the case PEEK's scheduler targets. These parameters were fixed before we measured any trace, and they differ from the two trace sources of §6. Our mean tool time of 5 s lies between theirs (1.40 and 8.51 s), but their medians are 15–33 times shorter than ours, 63–85% of their tool times are under 1 s against 18% of ours, and their coefficients of variation are 4.3–4.7 against 1. Their outputs per call are 2.7–5.9 times our 100 tokens, and TraceLab's mean prompt, 172k tokens, exceeds our 49,152-token context limit. Apart from one secondary mini-SWE-agent workload, the trace-driven runs of §6 replace only the tool-time law, and only in the simulator. With no shared prefix a miss reuses nothing from other sessions. A shared 2000-token system prompt, cached once, would cut the prompt work of a session from 154,529 to 125,007 tokens when every turn misses and from about 9,600 to 7,600 when every follow-up hits, so the ratio of the two, about 16, would barely change. Production agent traces report a hit rate of about 55% across turn boundaries and user idle gaps of minutes [45], while 93% of our turns hit in the good state (not the same metric). The simulator suggests that real tool-time laws keep the trap but move the window and the cap; the engine has not tested this. Our clients never retry or abandon a request, and the 7200 s request timeout never fired, although trapped turns took 163–237 s on average from send to completion at 300k and 321–511 s with the native pool. Retries would add load in the trapped state and abandonment would remove it, so either could change whether the engine recovers; neither was tested.

Few runs, finite horizons, nominal rates, uncontrolled factors. The closed curves and the from-empty sweeps have one run per point, so the peak's location (20 against 24 at 300k and on vLLM, 64 against 80 native) is within noise and single runs near the cliff land on either side of it. The registered tests, the grid and the vLLM runs share the same three seeds, so their runs are not independent across rates, pools or engines (§4.1). All rates in this paper are nominal. Each from-empty sweep used a single arrival seed, whose realized rates were 6–13% above nominal, and in the evaluation windows of the pre-registered SGLang tests the runs labelled 1.35 and 1.65 λ_{ref} received 1.27–1.45 and 1.61–1.75 λ_{ref} (native, 2.5 and 4.0: 2.48–2.73 and 3.89–4.28; vLLM BURST: 1.24–1.45 and 1.56–1.76). "Does not leave" means within 7500 s on the engine and 30,000 s in the simulator; the 3000 s horizon of the from-empty runs cannot see slow escapes, so the true edges may be below 2.0 λ_{ref} (both engines) and 5.5 λ_{ref} (native). The native cap rests on that coarse curve: $\mu(80)$ was within 0.35% of $\mu(64)$, and the cliff lies somewhere between 80 and 96 with no run in between. The registered smallest-within-5% rule gives 64 whichever of 64 and 80 is the true peak (the registration still flags the cap as ambiguous), and with no grid point between 48 and 64 it could not have found a smaller cap; a rule that takes the argmax could as well have picked 80, which may sit within one grid step of the cliff; how far a cap of 64 is from the cliff was not measured. The registered rule loses margin at both ends of the window: the slope test near λ_{miss}, and the token test at high multiples, where the trapped engine's prefill rate rises above the all-miss rate (1.106–1.124 times it at 4.0 λ_{ref} with the native pool, against a tolerance of 1.15), so at somewhat higher rates the rule would stop calling a stuck run trapped. In both SGLang closed curves the cliff coincides with a change of job and node (at 300k between $n = 32$ and 48, native between 80 and 96; Table 3), so neither curve excludes a node or job effect on where the cliff falls. The collapsed side ran on different nodes in the two curves, which argues against one slow node; on vLLM the hit fraction crossed 0.5 between two runs of one job on one node ($n = 28$ and 32); and both registered SGLang tests, which ran on all three nodes with consistent outcomes, argue against a node effect on the trap. The SGLang server seed was not pinned, and the load client runs on the server's node.

Model and simulators. Eq. (1) is used only for the window, the recovery threshold and the capped drain time, which the engine met within −4% to +52% given the population present at the burst, and it is not quantitative in congestion (§3.3). Proposition 1 is a limit statement about the closed-population throughput $\mu(n)$ as n grows and says nothing about any finite population. It covers request-level FCFS and, on our reading of the engines' sources, vLLM's FCFS with its preemption and SGLang's stock LPM, which stops re-sorting its queue when more than 128 requests wait; it does not cover program-level FCFS, pure resident-first orders or eviction rules that foresee tool times. It assumes that long-run rates exist, private prefixes, at most one outstanding request per session, no offloading of evicted KV, i.i.d. tool times with finite mean that are independent of everything else and unknown to the engine in advance, a bounded number of running requests, a bounded admission rate, and an engine that in congestion computes prompt tokens no faster than with caching disabled; it gives equality only if the engine computes them as fast. With zero hits our engine computed them at 1.03 times the all-miss rate near the stationary turn mix and at 0.99–1.12 times it over the windows we measured, so (A3) and (A4) hold on it only to within a few percent at the stationary mix and to within 12% over those windows. The proof and Lemma 1 were checked by AI review agents and have not been refereed. The open system's recovery threshold rests on a heuristic work balance and, without a monotone tail of μ, can lie below λ_{miss}; that recovery from deep congestion without a cap needs a rate below

λ_{miss} is untested on the engine; pure LPM's congested rate is a conjecture; and the capped system's stochastic stability is not proved. The calibrated simulator failed its level test at $n = 48$, reserves whole turns, omits partial prefixes and the refresh, frees a finished session's KV at once (an oracle: a real engine keeps it as evictable cache), and understated LPM. In its primary mode it put the native from-empty edge at least one grid step too high in nominal multiples, against a single real run whose arrivals exceeded the real peak, and for vLLM it predicted a later, gentler cliff than the engine showed (hit fraction 0.46–0.53 against 0.08 at $n = 32$). The mechanism comparison uses an older simulator whose assumed costs are about 1.5 times the fitted ones, so its absolute rates and recovery thresholds do not carry over to the engine. Its pin rows are fixed, configured TTLs, not Continuum's cost model; its PEEK rows are ablations and a projection of the released scheduler; and its CONCUR row is a surrogate of the paper's control equation.

11 Conclusion

Prefix caching turns multi-turn agent serving into a thrashing workload: the cache that makes follow-up turns cheap belongs to sessions idle in tool calls or waiting in the queue, and admission reclaims it. In our SGLang configuration the engine stayed in the good state from empty over 3000 s at nominal rates up to twice the all-miss capacity (2.2 times in realized arrivals), and at nominal rates of 1.35 and 1.65 times that capacity the good state was metastable: in a pre-registered test a burst moved the engine into an all-miss state that it did not leave within 7500 s, the no-burst arm stayed good and the capped arm recovered, and a simulator whose cost model was fitted only to server-log timing had predicted this outcome in predictions pushed to GitHub before the test was submitted. With the KV pool SGLang sizes itself, the cliff and the from-empty edge moved out, the edge to between nominal 5.5 and 6.5 times the all-miss capacity, and a second pre-registered test at 2.5 and 4.0 times passed at both rates with every run in its predicted state, as the simulator had predicted for somewhat higher rates. On vLLM 0.32.0 with the 300k-token pool, the engine's own closed curve and from-empty edge select the same window and cap as on SGLang; in exploratory runs without control arms it reached the same all-miss state after the same burst, and a registered paired test with all three arms was still running when this version was prepared (§5.7). These results are for one model, one GPU type and synthetic sessions with exponential 5 s tool times and no shared prefixes. Operators should measure the closed-population curve for the workload they serve, cap admitted sessions at the smallest population within a few percent of its peak, accepting that a burst then waits at the client, and re-measure when the workload changes.

Code and data availability

The repository `https://github.com/AkshatValse/Nova-Research` holds the code (simulator, load generator, analysis and Slurm job scripts), the experiment configurations, the registration files (`configs/agentload_prereg_*.yaml`), the frozen blind predictions (`notes/PREDICTIONS*.md`, with their tags), all real-engine data with the job logs (`results/nova/`), the research log (`notes/LOG.md`), the scripts behind every figure and derived table, and the source of every number in the paper (`paper/NUMBERS.md`); this version corresponds to tag `arxiv-v1`. The directory `provenance/` holds the archived GitHub push log (`github_activity_2026-10.json`), which is the external record of when each registration and prediction reached GitHub (§4.3, Appendix C), the cluster's accounting records of every cited job (`sacct_cited_jobs_2026-10-30.txt`), and the evidence of the independent verifier of the first prediction round. Simulator outputs are not stored, apart from their summaries; each can be regenerated from the command, commit and configuration recorded with the run (`meta.json`, `config.yaml`). The derived tables in `paper/derived/` name each engine run by its path on the cluster, `results/agentload_*/<timestamp>`; the run is stored under the pull directory of its jobs in `results/nova/`, with the same final path components. The agents' tool-call transcripts (§4.4, Appendix C) are not in the repository.

Use of AI tools

LLM coding agents (Anthropic Claude through Claude Code, including Claude Sonnet subagents, and OpenAI Codex sessions) were used extensively under the authors' direction. They wrote and tested much of the simulator, load generator and analysis code; prepared and submitted the cluster jobs, under authorizations the first author gave for each experiment or set of experiments; made the blind simulator predictions (§4.4); specified the prediction rounds and, afterwards, checked the first round's tool-call transcripts; made, in a separate Codex session, the independent verification of the first round, which reused the prediction agents' copy of the repository rather than a fresh one; audited the citations; read the engine sources, partly through a web tool that returns a language model's summary of each page; checked the model and reviewed the proofs; and drafted and revised the text. The proofs (§3.4, Appendix E) were checked by AI review agents and have not been refereed. Every commit, including those the agents made, is recorded under the first author's identity; a history rewrite on 2026-10-22 removed the agents' co-author trailers from commit messages and changed nothing else (`provenance/commit-map.tsv` maps old hashes to new ones). The authors chose the research question and the experiments and take full responsibility for the content.

Acknowledgments

The experiments ran on the Nova cluster at Iowa State University. The research reported in this paper is partially supported by the HPC@ISU equipment at Iowa State University, some of which has been purchased through funding provided by NSF under MRI grants number 1726447 and MRI2018594.

References

[1] Reyna Abhyankar, Zijian He, Vikranth Srivatsa, Hao Zhang, and Yiying Zhang. InferCept: Efficient intercept support for augmented large language model inference. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 81–95, 2024.

[2] Peter Alvaro, Rebecca Isaacs, Rupak Majumdar, Kiran-Kumar Muniswamy-Reddy, Mahmoud Salamati, and Sadegh Soudjani. Formal analysis of metastable failures in software systems. arXiv:2510.03551, 2025.

[3] Ruicheng Ao, Gan Luo, David Simchi-Levi, and Xinshang Wang. Optimizing LLM inference: Fluid-guided online scheduling with memory constraints. arXiv:2504.11320 (v4, 13 June 2026), 2025.

[4] Ruicheng Ao, Jing Dong, Gan Luo, and David Simchi-Levi. Service-induced congestion in memory-constrained LLM serving. arXiv:2606.15555 (v3, 20 October 2026), 2026.

[5] Mayank Arya, Agrim Bari, Anjaly Parayil, Yogesh Simmhan, Renee St. Amant, Victor Rühle, and Saravan Rajmohan. Understanding KV residency in production-scale agentic LLM workloads. arXiv:2610.22863 (v1, 16 October 2026), 2026.

[6] L. A. Belady. A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal*, 5(2): 78–101, 1966.

[7] Zhuohang Bian, Feiyang Wu, Zhuoran Li, Teng Ma, and Youwei Zhuo. TokenCake: A KV-cache-centric serving framework for LLM-based multi-agent applications. arXiv:2510.18586; to appear at EuroSys '27, 2025.

[8] Anish Biswas, Kanishk Goel, Srivarshinee S, Jayashree Mohan, Alind Khare, Anjaly Parayil, Ramachandran Ramjee, and Chetan Bansal. Sutradhara: An intelligent orchestrator-engine co-design for tool-based agentic inference. arXiv:2601.12967 (v3, 22 April 2026), 2026.

[9] Nathan Bronson, Abutalib Aghayev, Aleksey Charapko, and Timothy Zhu. Metastable failures in distributed systems. In *Proceedings of the Workshop on Hot Topics in Operating Systems (HotOS '21)*, Ann Arbor, MI, 2021. doi: 10.1145/3458336.3465286.

[10] Shelby L. Brumelle. On the relation between customer and time averages in queues. *Journal of Applied Probability*, 8(3):508–520, 1971. doi: 10.2307/3212174.

[11] Shiyi Cao, Yichuan Wang, Ziming Mao, Pin-Lun Hsu, Liangsheng Yin, Tian Xia, Dacheng Li, Shu Liu, Yineng Zhang, Yang Zhou, Ying Sheng, Joseph Gonzalez, and Ion Stoica. Locality-aware fair scheduling in LLM serving. arXiv:2501.14312, 2025.

[12] Qiaoling Chen, Zhisheng Ye, Tian Tang, Peng Sun, Boyu Tian, Guoteng Wang, Shenggui Li, Yonggang Wen, Zhenhua Han, and Tianwei Zhang. CONCUR: High-throughput agentic batch inference of LLM via congestion-based concurrency control. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 17881–17891, 2026. arXiv:2601.22705.

[13] Hong-Tai Chou and David J. DeWitt. An evaluation of buffer management strategies for relational database systems. In *Proceedings of VLDB 85*, pages 127–141, Stockholm, 1985.

[14] Neil Chowdhury, James Aung, Jun Shern Chan, Oliver Jaffe, Dane Sherburn, Giulio Starace, Evan Mays, Rachel Dias, Marwan Aljubeh, Mia Glaese, Carlos E. Jimenez, John Yang, Leyton Ho, Tejal Patwardhan, Kevin Liu, and Aleksander Madry. Introducing SWE-bench Verified. OpenAI blog post, 13 August 2024 (updated 24 February 2025); `https://openai.com/index/introducing-swe-bench-verified/`, 2024.

[15] P. J. Courtois. Decomposability, instabilities, and saturation in multiprogramming systems. *Communications of the ACM*, 18(7):371–377, 1975. doi: 10.1145/360881.360887.

[16] J. G. Dai, Tianze Deng, Yueying Li, and Tianyi Peng. Throughput-optimal scheduling algorithms for LLM inference and AI agents. arXiv:2504.07347 (v3, 18 May 2026), 2025.

[17] Peter J. Denning. Thrashing: Its causes and prevention. In *Proceedings of the AFIPS Fall Joint Computer Conference*, pages 915–922, 1968.

[18] Peter J. Denning. The working set model for program behavior. *Communications of the ACM*, 11(5):323–333, 1968. doi: 10.1145/363095.363141.

[19] Peter J. Denning, Kevin C. Kahn, Jacques Leroudier, Dominique Potier, and Rajan Suri. Optimal multiprogramming. *Acta Informatica*, 7(2):197–216, 1976. doi: 10.1007/BF00265771.

[20] Ishan Dhanani, Jamie Li, and Karen Chung. Session-aware agentic inference with NVIDIA Dynamo. PyTorch Blog, 8 October 2026; `https://pytorch.org/blog/session-aware-agentic-inference-with-nvidia-dynamo/`, 2026.

[21] Zhuolun Dong and Junyu Cao. Flow-controlled scheduling for LLM inference with provable stability guarantees. arXiv:2604.11001, 2026.

[22] Ali Farahbakhsh, Andreas Haeberlen, Qingjie Lu, Lorenzo Alvisi, Robbert van Renesse, and Shir Cohen. Modeling metastability. In *Proceedings of the 24th ACM Workshop on Hot Topics in Networks (HotNets '25)*, pages 19–24, College Park, MD, USA, 2025. ACM. doi: 10.1145/3772356.3772426.

[23] Ali Farahbakhsh, Qingjie Lu, Lorenzo Alvisi, Andreas Haeberlen, and Robbert van Renesse. Characterizing metastable faults and failures. arXiv:2606.00942 (v2, 2 June 2026), 2026.

[24] Guy Fayolle, Erol Gelenbe, and Jacques Labetoulle. Stability and optimal control of the packet switching broadcast channel. *Journal of the ACM*, 24(3): 375–386, 1977. doi: 10.1145/322017.322019.

[25] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. Cost-efficient large language model serving for multi-turn conversations with CachedAttention. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, pages 111–126, Santa Clara, CA, 2024. USENIX Association. arXiv:2403.19708.

[26] R. J. Gibbens, P. J. Hunt, and F. P. Kelly. Bistability in communication networks. In G. R. Grimmett and D. J. A. Welsh, editors, *Disorder in Physical Systems*, pages 113–127. Oxford University Press, 1990.

[27] Farzad Habibi, Tania Lorido-Botran, Ahmad Showail, Daniel C. Sturman, and Faisal Nawab. MSF-Model: Queuing-based analysis and prediction of metastable failures in replicated storage systems. In *Proceedings of the 43rd International Symposium on Reliable Distributed Systems (SRDS)*, pages 12–22, Charlotte, NC, USA, 2024. IEEE. doi: 10.1109/SRDS64841.2024.00013.

[28] Hans-Ulrich Heiss and Roger Wagner. Adaptive load control in transaction processing systems. In *Proceedings of the 17th International Conference on Very Large Data Bases (VLDB '91)*, pages 47–54, Barcelona, 1991. Morgan Kaufmann.

[29] Daniel P. Heyman and Shaler Stidham, Jr. The relation between customer and time averages in queues. *Operations Research*, 28(4):983–994, 1980. doi: 10.1287/opre.28.4.983.

[30] Zetao Hong, Song Yuan, Yuanhao Ding, Yibo Zhu, Daxin Jiang, Zhibin Wang, and Chen Tian. Scheduling mixed RL rollouts beyond prefix locality. arXiv:2608.11152 (v1, 11 August 2026), 2026.

[31] Lexiang Huang, Matthew Magnusson, Abishek Bangalore Muralikrishna, Salman Estyak, Rebecca Isaacs, Abutalib Aghayev, Timothy Zhu, and Aleksey Charapko. Metastable failures in the wild. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 73–90, 2022.

[32] Rebecca Isaacs, Peter Alvaro, Rupak Majumdar, Kiran-Kumar Muniswamy-Reddy, Mahmoud Salamati, and Sadegh Soudjani. Analyzing metastable failures. In *Proceedings of the Workshop on Hot Topics in Operating Systems (HotOS '25)*, pages 172–178, Banff, AB, Canada, 2025. ACM. doi: 10.1145/3713082.3730380.

[33] Van Jacobson. Congestion avoidance and control. In *Symposium Proceedings on Communications Architectures and Protocols (ACM SIGCOMM '88)*, pages 314–329, 1988. doi: 10.1145/52324.52356.

[34] Song Jiang and Xiaodong Zhang. LIRS: An efficient low inter-reference recency set replacement policy to improve buffer cache performance. *ACM SIGMETRICS Performance Evaluation Review*, 30(1):31–42, 2002. doi: 10.1145/511399.511340.

[35] Youhe Jiang, Fangcheng Fu, Binhang Yuan, Krishna Malladi, Ehsan K. Ardestani, Zhan Shu, Adnan Aziz, and Yi Xu. Coda: Exploiting admission flexibility for coding-agent serving. arXiv:2610.03088 (v1, 2 October 2026), 2026.

[36] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR 2024)*, 2024.

[37] Hao Kang, Ziyang Li, Weili Xu, Xinyu Yang, Yinfang Chen, Junxiong Wang, Beidi Chen, Tushar Krishna, Chenfeng Xu, and Simran Arora. ThunderAgent: A fast, simple, and program-aware agentic inference system. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 55630–55657, 2026. arXiv:2602.13692.

[38] Jinpyo Kim, Mingi Kwon, Younghoon Min, Jongryool Kim, and Jishen Zhao. Dynamic HBM repartitioning for multi-turn MoE serving. arXiv:2609.13537 (v1, 11 September 2026), 2026.

[39] Leonard Kleinrock and Simon S. Lam. Packet switching in a multiaccess broadcast channel:

Performance evaluation. *IEEE Transactions on Communications*, 23(4):410–423, 1975. doi: 10.1109/TCOM.1975.1092814.

[40] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*, pages 611–626. ACM, 2023. doi: 10.1145/3600006.3613165.

[41] Hanchen Li, Runyuan He, Qiuyang Mang, Qizheng Zhang, Huanzhi Mao, Xiaokun Chen, Hangrui Zhou, Huanchen Zhang, Alvin Cheung, Joseph Gonzalez, and Ion Stoica. Continuum: Efficient and robust multi-turn LLM agent scheduling with KV cache time-to-live. arXiv:2511.02230 (v7, 8 September 2026), 2025.

[42] Luchang Li, Shuaishuai Wang, Zhao Ruan, Dongfang Li, and Bozhao Gong. The KV cache working set: Online capacity planning for LLM inference systems. arXiv:2609.27746 (v1, 23 September 2026), 2026.

[43] Shuhuai Li, Shengdong Ye, Dongdong Ge, Yinyu Ye, and Jianghao Lin. TAO: Tool-aware joint scheduling for LLM-based agentic serving. arXiv:2610.37230 (v1, 27 October 2026), 2026.

[44] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. Parrot: Efficient serving of LLM-based applications with semantic variable. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 929–945, Santa Clara, CA, 2024. USENIX Association. arXiv:2405.19888.

[45] Banruo Liu, Haoran Qiu, Íñigo Goiri, Rodrigo Fonseca, Ricardo Bianchini, and Esha Choukse. Agentic coding in the wild: Characterizing GitHub Copilot traces at production scale. arXiv:2608.00101 (v1, 30 July 2026), 2026.

[46] Yipeng Liu, Yingqiang Zhang, Feifei Li, and Huanchen Zhang. Ask the tool, don't guess: Agent tool calls hold their progress, and the serving system should read it. arXiv:2609.18849, 2026.

[47] Yiyu Liu, Minlan Yu, and Juncheng Yang. When fancy eviction fails: Rethinking cache replacement for LLM prefix reuse. arXiv:2609.28870 (v2, 1 October 2026), 2026.

[48] Michael Luo, Xiaoxiang Shi, Colin Cai, Tianjun Zhang, Justin Wong, Yichuan Wang, Chi Wang, Yanping Huang, Zhifeng Chen, Joseph E. Gonzalez, and Ion Stoica. Agentix: An efficient serving engine for LLM agents as general programs. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*, pages 2443–2459, Renton, WA, 2026. USENIX Association. arXiv:2502.13965 (as Autellix).

[49] Jiaying Meng and Bojie Li. Metronome: Bound the cache, keep the beat for real-time interaction model serving. arXiv:2607.02640 (v1, 2 July 2026), 2026.

[50] Utopia Meng, Unicornt Zhao, Derek Li, Goalen Gao, and Frank Du. Talaria: Session-aware serverless serving of hundred-billion-parameter LLMs. arXiv:2607.17181 (v1, 19 July 2026), 2026.

[51] John Nagle. Congestion control in IP/TCP internetworks. RFC 896, 6 January 1984; `https://www.rfc-editor.org/rfc/rfc896`, 1984.

[52] Chengyi Nie, Nian Si, and Zijie Zhou. A queueing-theoretic framework for stability analysis of LLM inference with KV cache memory constraints. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 93016–93031, 2026. arXiv:2605.04595.

[53] Zaifeng Pan, Ajjkumar Dahyalal Patel, Yipeng Shen, Zhengding Hu, Yue Guan, Wan-Lu Li, Lianhui Qin, Yida Wang, and Yufei Ding. KVFlow: Efficient prefix caching for accelerating LLM-based multi-agent workflows. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, pages 139912–139931, 2025. doi: 10.52202/085713-4208. arXiv:2507.07400.

[54] Zaifeng Pan, Yipeng Shen, Zhengding Hu, Zhuang Wang, Aninda Manocha, Zheng Wang, Zhongkai Yu, Yue Guan, and Yufei Ding. ScaleSim: Serving large-scale multi-agent simulation with invocation distance-based memory management. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 95898–95913, 2026. arXiv:2601.21473.

[55] Zaifeng Pan, Chris Wu, Zhengding Hu, Xinwei Qiang, Zhongkai Yu, and Yufei Ding. RR-Evict: Fine-grained prefix cache eviction beyond LRU for agentic LLM serving. arXiv:2609.32278 (v2, 2 October 2026; under review), 2026.

[56] Sungjoon Park, Changue Jung, Kyungno Joo, Mincheol Kang, Jaehyung Ahn, Sehwan Lee, and Sangjoon Kim. ServeTwin: A benchmark-validated simulator for distributed LLM architecture exploration. arXiv:2610.02732 (v1, 2 October 2026), 2026.

[57] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The Berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large

language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 48371–48392, 2025.

[58] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Trading more storage for less computation — A KVCache-centric architecture for serving LLM chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*, pages 155–170, Santa Clara, CA, 2025. USENIX Association. Technical-report version: arXiv:2407.00079, titled "Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving".

[59] Qwen Team. Qwen2.5 technical report. arXiv:2412.15115, 2024.

[60] Rakibul Hasan Rajib, Mengxin Zheng, and Qian Lou. AgentServeSim: Serving-system simulation and policy search for LLM agent programs. arXiv:2606.09613 (v3, 5 September 2026), 2026.

[61] Bianca Schroeder, Mor Harchol-Balter, Arun Iyengar, Erich Nahum, and Adam Wierman. How to determine a good multi-programming level for external scheduling. In *Proceedings of the 22nd International Conference on Data Engineering (ICDE '06)*, page 60, 2006. doi: 10.1109/ICDE.2006.78.

[62] Kunming Shao, Jierun Chen, Jiangnan Yu, Xiao-Hui Li, Chaofan Tao, Yanli Wang, Huanxin Lin, Kwang-Ting Cheng, Chi Ying Tsui, and Haoli Bai. EfficientAgent: What makes KV cache offloading work for concurrent agents? arXiv:2609.33762 (v1, 27 September 2026), 2026.

[63] Yannis Smaragdakis, Scott Kaplan, and Paul Wilson. EELRU: Simple and effective adaptive page replacement. In *Proceedings of the 1999 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*, pages 122–133, 1999. doi: 10.1145/301453.301486.

[64] SWE-agent. mini-SWE-agent. GitHub repository, release v2.0.0 (11 February 2026); `https://github.com/SWE-agent/mini-swe-agent`, 2026.

[65] Shubham Tiwari, Tapan Chugh, Nash Rickert, Simon Peter, Ratul Mahajan, and Haiying Shen. CacheWise: Understanding workloads and optimizing KVCache management for efficiently serving LLM coding agents. arXiv:2606.16824, 2026.

[66] Michael Wang, Yikang Yue, Shaobo Li, Yirui Eric Zhou, Chen Wang, and Jian Huang. Benchmarking LLM serving systems for agentic AI workloads with XPerf. arXiv:2608.20370, 2026.

[67] Yifei Wang, Hancheng Ye, Yechen Xu, Cong Guo, Chiyue Wei, Qinsi Wang, Dongting Li, Tingjun Chen, Hai "Helen" Li, Danyang Zhuo, and Yiran Chen. MARS: Efficient, adaptive co-scheduling for heterogeneous agentic systems. arXiv:2604.26963 (v2, 15 June 2026), 2026.

[68] Fangzhou Wu, Sandeep Silwal, and Qiuyi (Richard) Zhang. Randomization boosts KV caching, learning balances query load: A joint perspective. In *International Conference on Learning Representations (ICLR 2026)*, 2026. arXiv:2601.18999.

[69] Tian Xia, Hanchen Li, Zhifei Li, Xiaokun Chen, Hao Kang, Yifan Qiao, Yi Xu, and Ion Stoica. Idleness is relative: Exploiting tool-call idle windows for offloading in agentic systems with MORI. arXiv:2606.00866, 2026.

[70] Bing Xie, Zhipeng Wang, Masahiro Tanaka, and Zhen Zheng. PEEK: Predictive queue-informed KV cache management for LLM serving. arXiv:2607.02525 (v2, 12 September 2026; under review), 2026.

[71] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, pages 50528–50652, 2024. doi: 10.52202/079017-1601. arXiv:2405.15793.

[72] Heyuan Yao, Chutong Gao, Yuan Lyu, Izzy Grosof, and David Simchi-Levi. Multi-turn LLM conversations under the least-recently-used policy: Mean-field asymptotics and hit ratio approximation. arXiv:2609.02027, 2026.

[73] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for Transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538, Carlsbad, CA, 2022. USENIX Association.

[74] Lingfan Yu, Jinkun Lin, and Jinyang Li. Stateful large language model serving with Pensieve. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25)*, pages 144–158. ACM, 2025. doi: 10.1145/3689031.3696086. arXiv:2312.05516.

[75] Yichao Yuan, Ankita Nayak, Souvik Kundu, and Nishil Talati. Agentic AI workload characterization. In *Proceedings of the 2026 IEEE International Symposium on Workload Characterization (IISWC)*, 2026. arXiv:2605.26297 (v2, 21 September 2026), listed there as "Agentic AI Workload Characteristics".

[76] Rui Zhang, Chaeeun Kim, Shaoting Feng, Kuntai Du, Yuhan Liu, Yi Zhong, Cheng-Wei Ching, Junchen Jiang, and Liting Hu. Learning agent execution for KV-cache management in agentic serving. arXiv:2608.14624, 2026.

[77] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, pages 62557–62583, 2024. doi: 10.52202/079017-2000.

[78] Kan Zhu, Mathew Jacob, Chenxi Ma, Yi Pan, Stephanie Wang, Arvind Krishnamurthy, and Baris Kasikci. TraceLab: Characterizing coding agent workloads for LLM serving. arXiv:2606.30560 (v2, 30 June 2026), 2026. Trace release v0.0.2, 24 July 2026: `https://github.com/uw-syfi/TraceLab`.

A Protocol details

Table 3: Real-engine stages (SLURM job IDs on Nova; submit, start, end and node of every job in `provenance/sacct_cited_jobs_2026-10-30.txt`).

Stage	Jobs	What it measures
Stage 0	17060165	smoke test
Stage C	17062103	closed $\mu(n)$, 300k pool, FCFS, $n \in \{16, 20, 24, 32\}$ (node gpu-1)
	17062104	FCFS, $n \in \{48, 96, 300\}$ (gpu-3)
	17062105	LPM, $n \in \{48, 300\}$ (gpu-2)
E0a	17072634–37	λ_{miss}, cache off
E0b	17084750–55	from-empty stability
E2	17093387–404	registered paired test, 2 rates × 3 seeds × 3 arms
native C	17096856	$\mu(n)$, native pool, $n \in \{24, 48, 64, 80\}$ (gpu-2)
	17096857	$n \in \{96, 128, 300\}$ (gpu-1)
native E0a	17096858	λ_{miss}, native pool
grid	17097086–103	BURST arm, 3 policy cells
vLLM	17097972–74	install, GPU check, smoke
native E0b	17100938–39	from-empty stability, native pool
vLLM E0a	17100940	λ_{miss}, vLLM, cache off
native E2	17102173–90	registered paired test, native pool, 2 rates × 3 seeds × 3 arms
vLLM BURST	17102318–23	BURST arm, vLLM, exploratory
vLLM C	17162959	$\mu(n)$, vLLM, 300k pool, $n \in \{16, 20, 24\}$ (gpu-3)
	17162960	$n \in \{28, 32, 40\}$ (gpu-3)
	17162961	$n \in \{48, 96, 300\}$ (gpu-2)
vLLM E0b	17162962–63	from-empty stability, vLLM
vLLM E2	17166121–38	registered paired test, vLLM, 2 rates × 3 seeds × 3 arms

Table 4 lists the workload and engine settings of the real-engine runs.

Stage 0. The smoke test (job 17060165) verified that every engine flag exists; that cached tokens are reported (a repeated 64-token prompt had 63 tokens cached); that the server returns output token ids; that contexts up to 40,350 tokens are accepted; that the KV pool is exactly 300,000 tokens; that client and server agree on prompt length (0 mismatches over 3191 requests); and that the flush empties the cache (canary 1023 cached tokens before, 0 after).

Stage C and E0a. Closed population; all sessions start at $t = 0$ and a finished session is replaced at once; 900 s warmup and an 1800 s measured window; one run per n (seed 0); flush and canary check between runs. The native-pool and vLLM closed curves used the same protocol, with the populations of Table 3. E0a disables the radix cache and runs 32 sessions with seeds 0, 1 and 2, plus 64 sessions with seed 0 as a saturation check. Its per-run values over the window were (sessions/s, turns-based sessions/s, uncached tokens/s): seed 0, 0.0567, 0.0643, 11,076; seed 1, 0.0817, 0.0773, 11,342; seed 2, 0.0767, 0.0730, 11,312; 64 sessions, 0.0544, 0.0676, 11,431. The spread comes from in-flight composition: the mean turn index of the requests in the window was 15.9, 13.2 and 14.1 for seeds 0, 1 and 2. vLLM E0a ran the same protocol with vLLM's prefix caching off: seed 0, 0.0539, 0.0603, 10,208; seed 1, 0.0728, 0.0705, 10,487; seed 2, 0.0700, 0.0662, 10,352; 64 sessions, 0.0467, 0.0631, 10,531.

Table 4: Workload and engines of the real-engine runs.

First prompt L_0	2000 tokens
Tool output per turn Δ	550 tokens
Output per turn d	100 tokens, greedy, `ignore_eos`
Turns T	$\min(\text{Geometric}(1/15), 60)$, $E[T] = 14.761$
Tool time Z	exponential, mean 5 s
Prompt at turn k	$2000 + 650(k-1)$ tokens
Output feedback	server-returned token ids
Engine	SGLang 0.5.22, FlashInfer
Scheduling	FCFS, waiting-prefix refresh on
Batching	chunked prefill 8192, ≤ 256 running
KV pool	300,000 tokens; native 955,757
Context limit	49,152 tokens
Model	Qwen2.5-7B-Instruct, rev. a09a3545
GPU	A100 80GB PCIe, 81,920 MiB, driver 580.178.04
vLLM	0.32.0, FCFS (default), prefix caching on chunked prefill 8192, ≤ 256 running 300,000 tokens (18,750 blocks of 16)

E0b. Poisson session arrivals from an empty cache and queue at 1.25, 1.5, 1.75, 2.0, 2.5 and 3.0 λ_{ref}; one 3000 s run per rate; stable if the final queue is below 50 and the fitted rise of the queue over the last 1500 s is at most 10. All runs used arrival seed 0, so they share one random-number stream and the arrivals at each rate are a subset of those at the next higher rate. The six runs received 302, 359, 416, 472, 585 and 697 sessions, 8.3–12.6% above λT and about 2.1 Poisson standard deviations above it in every run. Native E0b used the same rule at 1.5, 2.5, 3.5, 4.5, 5.5 and 6.5 times the native λ_{ref} (361, 587, 811, 1034, 1257 and 1478 sessions, 5.6–11.7% above λT), and vLLM E0b the multiples of the 300k runs of vLLM's λ_{ref} (279, 331, 383, 435, 539 and 642 sessions, 8.6–13.3% above λT; stable through 2.0, final queue 81.6 and 247.1 at 2.5 and 3.0).

E2. One arm per 8700 s job, each on a fresh server; burst of 200 fresh sessions within 60 s at $t_b = 1200$ s for BURST and CAPPED; CAPPED gate $n_{\text{cap}} = 20$; evaluation window $[t_b + 4500, 8700]$ s. The registered rule (Table 5) was applied by the analysis tool frozen at the registration commit, with the same parameter overrides that the jobs themselves used. Native E2 used the same protocol with the native registration's rates, reference rates and gate $n_{\text{cap}} = 64$. Twelve of its 18 runs started after the cluster checkout moved from the registration commit to `a78ce07`, which adds only the vLLM registration, its input file and two notes files (the research log and the status notes), with no code change, and its verdict ran on that commit; two of its 18 servers sized the pool one token smaller, at 955,756. The vLLM paired test uses the same protocol with

the vLLM registration and $n_{\text{cap}} = 20$. It uses the same seeds, rates and reference rate as the exploratory vLLM BURST runs, so its BURST arm repeats their workloads rather than replicating them independently.

Table 5: The registered decision rule.

Rate estimator	uncached prefill tokens/s
Reference rate	E0a all-miss, 11,243 tokens/s (native: 11,282; vLLM: 10,349)
λ_{miss} in the rule	the cell's λ_{ref}
Rate tolerance	±0.15
Backlog slope	least squares over 10 s bin means of sessions in the system, gate included
Minimum window	600 s
Trapped	$\lambda > \lambda_{\text{miss}}$, rate within tolerance, and slope > 0.5 $(\lambda - \lambda_{\text{miss}})$
Recovered	hit > 0.8 and outstanding < 20, last 300 s
Draining	slope < 0
Indeterminate	none of the above
Precedence	trapped, recovered, draining, indeterminate
Hit metric	turns with cached/prompt > 0.5
Pass	≥ 2/3 of ≥ 3 seeds with all three arm conditions met in the seed

Backlog definitions. The figures and Table 1 report the backlog as sessions in the system (arrived minus ended), which includes sessions asleep in tool calls (8–12 in the good state at 300k, 16–34 with the native pool). The registered RECOVERED test uses the outstanding backlog (requests in flight plus sessions held at the gate). The simulator's predictions count waiting requests plus the gate queue.

B The registrations

The registration is `configs/agentload_prereg_2026-10-24.yaml`, status FILLED, committed in `d279b7b` at 15:09:12 CDT on 2026-10-24 and pushed to GitHub at 15:09:54, before the E2 jobs were submitted at 15:13:39, SHA-256 `2a6d680a9615bba962a5673d6db6b3e4 ca5102bf778ca974cea46147f5cc0ac6`. Its fields:

- **Cell and inputs.** Engine cell `sglang-fcfs-r1` (SGLang FCFS, waiting-prefix refresh on); committed copies of the Stage C curve, the E0a calibration and the E0b bracket under `prereg/inputs/2026-10-23/`; protocol 8700 s, burst of 200 within 60 s at 1200 s.
- **Measured.** $\mu(n)$ in sessions/s: 0.1756 ($n = 16$), 0.2239 (20), 0.2333 (24), 0.1150 (32), 0.0489 (48), 0.0611 (96), 0.1078 (300). $\mu^* = 0.2333$; $n_{\text{cap}} = 20$ by the smallest-within-5% rule (argmax 24; flagged ambiguous). $\lambda_{\text{miss}} = 0.07167$ (sessions) and 0.07153 (turns-based, $= \lambda_{\text{ref}}$); all-miss rate 11,243 tokens/s; E0b largest all-stable multiple 2.0, first unstable 2.5, monotone; hardware A100 80GB PCIe.
- **Predicted boundaries.** Recovery threshold λ_{ref} (not $\mu(300)$, which a synchronized start biases upward). Window (0.0715, 0.1431) sessions/s = (1.0, 2.0) λ_{ref}, upper edge from E0b. CAPPED drain must fit within 80% of the 7200 s available after the burst (5760 s), which holds for λ up to 2.64 λ_{ref}; predicted drain 1571 s at 1.35 and 1889 s at 1.65.
- **Rates.** 1.35 and 1.65 λ_{ref} by the one-third/two-thirds rule, rounded to 0.05, inside the margin band (1.1, 1.8); seeds 0, 1, 2. On 2026-10-22 the runbook proposed the rate rule and a cap of 20, with keeping 24 as the alternative (`288b801`, 13:13 CDT); both rules were adopted in the project status notes at 13:50 (`4435c51`) and reached GitHub at 17:45, before E0b was submitted on 2026-10-23; the registration's own text dates them 2026-10-23.
- **Predictions.** Inside the window: BURST trapped, NOBURST good through the run horizon, CAPPED recovered if the drain fits. Below λ_{ref}: BURST not trapped. Above the window: NOBURST may escape, so no such rate was registered.
- **Kill conditions.** NOBURST not good in at least one third of the seeds at a registered rate; BURST not trapped in two thirds of the seeds at every registered rate; CAPPED not recovering while BURST traps and the drain fits; a window that is empty or narrower than the noise.
- **Warnings recorded at filling.** $|\mu(300)/\lambda_{\text{miss}} - 1| = 0.507$ (a check only), and the ambiguous cap.

The verdict is in `results/agentload_analysis_burst2/20261025-144012/` inside the E2 data directory: pre-registered, decisive pass at 1.35 and 1.65 λ_{ref}. The three grid cells have their own derived registrations, `configs/agentload_prereg_2026-10-24_grid_<cell>.yaml` for the cells `sglang-fcfs-r0`, `sglang-lpm-r1` and `sglang-lpm-r0`, with status EXPLORATORY; the analysis never calls them decisive.

The native registration. `configs/agentload_prereg_2026-10-25_native.yaml`, status FILLED, committed in `b8c19ec` at 17:31:16 CDT on 2026-10-25 and pushed to GitHub at 17:31:56, 10 minutes before the native E2 jobs were submitted (17:42:29), SHA-256 `21bf7c03e0630b65b80d086058815e12 c5ba9fbe41e9864d3dd0183b9bf776c1`; the same template, formulas, trap rule, kill conditions and protocol as above. Cell `sglang-fcfs-r1-native`; inputs under `prereg/inputs/2026-10-25-native/`. Measured: $\mu(n)$ 0.2650 ($n = 24$), 0.4311 (48), 0.4789 (64), 0.4772 (80), 0.1228 (96), 0.0667 (128), 0.1006 (300); $\mu^* = 0.4789$; $n_{\text{cap}} = 64$ by the smallest-within-5% rule (also the argmax; flagged ambiguous, $n = 80$ within 0.35%); $\lambda_{\text{ref}} = 0.07180$ (turns-based); all-miss rate 11,282 tokens/s; native E0b largest all-stable multiple 5.5, first unstable 6.5, monotone. Window (0.0718, 0.3949) sessions/s = (1, 5.5) λ_{ref}, upper edge from native E0b, below $\mu^*/\lambda_{\text{ref}} = 6.67$. Rates 2.5 and 4.0 λ_{ref} (0.1795 and 0.2872 sessions/s) by the same rule, inside the margin band (1.1, 4.95); the rule was recorded in the project status notes (`6cb43e9`, pushed 15:08 CDT) before native E0b was read. Predicted drains 668 s and 1043 s; the drain fits up to 6.19 λ_{ref}. Warnings recorded at

filling: $|\mu(300)/\lambda_{\text{miss}} - 1| = 0.401$, the ambiguous cap, and an incomplete grid (no $n = 16$, 20 or 32). The verdict is `results/agentload_analysis_burst2/20261026-100653/` inside the native E2 data directory: pre-registered, decisive pass at 2.5 and 4.0 λ_{ref}.

The exploratory vLLM registration. `configs/agentload_prereg_2026-10-24_grid_vllm-fcfs.yaml`, status EXPLORATORY, derived from the E2 registration, committed in `a78ce07` at 18:06:18 CDT on 2026-10-25 and pushed to GitHub at 18:06:33, 46 s before the vLLM BURST jobs were submitted, SHA-256 `c5037cac6003c4e98da89f88dae953bf e68bb21a27a4e3e5e3dab3c8891aeab5`. Cell `vllm-fcfs`, BURST arm only; the window, multiples, protocol and trap rule are inherited from `sglang-fcfs-r1`, and the reference rates come from `prereg/inputs/2026-10-25-vllm/calibration.json`, a byte-identical copy of the vLLM E0a calibration. Its predicted-boundary block is copied unchanged from the E2 registration (window 0.0715–0.1431 sessions/s, all-miss rate 11,243 tokens/s), but the runs used vLLM's references, 0.06566 sessions/s and 10,349 tokens/s (`lambda_ref_resolved` and `uncached_tokens_ref_resolved` in each run's `meta.json`), so their rates were 1.35 and 1.65 times vLLM's λ_{ref}. The verdict, `results/agentload_analysis_burst2/20261026-100701/` inside the native E2 and vLLM BURST data directory, reports per-run states and no registered verdict.

The vLLM registration. `configs/agentload_prereg_2026-10-30_vllm.yaml`, status FILLED, SHA-256 `8f2d5b44d6365795fedde8bf673d51e5 4bf8a685a14d563c86de44492bb95f84`; the same template, formulas, trap rule, kill conditions and protocol as the E2 registration. Cell `vllm-fcfs`; inputs: the vLLM Stage C curve and E0b bracket under `prereg/inputs/2026-10-30-vllm/`, byte-identical to the analyses in the vLLM Stage C and E0b data directories, and the vLLM E0a calibration above. Measured: $\mu(n)$ 0.1750 ($n = 16$), 0.2194 (20), 0.2183 (24), 0.1189 (28), 0.0433 (32), 0.0411 (40), 0.0439 (48), 0.0572 (96), 0.1050 (300); $\mu^* = 0.2194$; $n_{\text{cap}} = 20$ by the smallest-within-5% rule (also the argmax; flagged ambiguous, $n = 24$ within 0.5%); $\lambda_{\text{ref}} = 0.06566$ (turns-based); all-miss rate 10,349 tokens/s; vLLM E0b largest all-stable multiple 2.0, first unstable 2.5, monotone. Window (0.0657, 0.1313) sessions/s $= (1, 2.0)\,\lambda_{\text{ref}}$, upper edge from vLLM E0b, below $\mu^*/\lambda_{\text{ref}} = 3.34$. Rates 1.35 and 1.65 λ_{ref} (0.0886 and 0.1083 sessions/s) by the same rule, inside the margin band (1.1, 1.8); this rule and the cap rule were recorded in the project status notes (`836063a`, pushed 12:16 CDT on 2026-10-30) after the vLLM Stage C and E0b jobs were submitted and before any of them had ended. Predicted drains 1529 s and 1800 s; the drain fits up to 2.81 λ_{ref}. Warnings recorded at filling: $|\mu(300)/\lambda_{\text{miss}} - 1| = 0.599$ and the ambiguous cap. The file was filled from five of the six vLLM E0b runs while the 3.0 λ_{ref} run was still running; with 2.5 λ_{ref} unstable that run could not change the edge, and it was unstable too, so the six-run bracket equals the registered one. The file was committed in `5a6fbcd` at 16:23:59 CDT, 37 s before the 18 jobs of the paired test (17166121–38) were submitted at 16:24:36 (the research log gives 16:30), and the jobs run from the cluster's copy of the repository at that commit; Slurm's record of the submission names the file but not its hash. It reached GitHub at 16:26:35, 1 min 59 s after the submission, when two of the jobs had started (at 16:25:42 and 16:26:17) and none could have ended. This registration therefore does not meet the definition of §4.3: it was committed before the jobs were submitted, but pushed after the first two had started, so we call this test registered, not pre-registered. As for E2, the job script writes the file's SHA-256 into every run's `meta.json`.

C Blind-prediction timeline

Commit and tag times are set by the committer. The external record is GitHub's activity log for the repository, archived in `provenance/github_activity_2026-10.json`, together with the cluster's accounting records of the cited jobs (`provenance/sacct_cited_jobs_2026-10-30.txt`); `provenance/push_times_2026-10.tsv` lists, for each registration and prediction commit, its first arrival on GitHub and the first job it concerns. GitHub records no push time for tags, so the push times given here are those of the first branch update that contains each commit. The branch holding B1–B4 (`64bc4e3`) and the verification reached GitHub at 14:53:07 CDT on 10-24, and the E2 registration (`d279b7b`) at 15:09:54, before the E2 jobs were submitted at 15:13:39. On 10-25, B1n–B4n (`e3ea48f`) arrived at 15:08:28, the native registration (`b8c19ec`) at 17:31:56 and the exploratory vLLM registration (`a78ce07`) at 18:06:33. Pull times are from our local records only.

B3 (E2) and B4 (native pool) reached GitHub before their experiments were submitted, so they are blind in time. B2 (E0b) was committed after E0b had finished on the cluster but before its results were pulled. The LLM agents that made the predictions worked in separate copies of the repository at a commit without the E0a or E0b results and were instructed not to contact the cluster and not to open the E0a or E0b result directories or the primary copy of the repository; they could technically have done so. Their tool-call transcripts, which are not in the repository, were checked afterwards by the orchestrating Claude Code session, which found no cluster command and no access to E0b before the predictions reached GitHub. For B2 this instruction-level control, checked after the fact, is the only safeguard. A scan of every line added between the commit the agents started from and the predictions found no use of withheld data, but a text scan cannot prove that nobody read it. B1 (E0a) rests on the same instruction only: E0a was committed before the predictions. The independent verifier, a separate OpenAI Codex session, had read the project status

Table 6: Order of events, 2026-10-21 to 10-26 (times CDT). Commits are in the repository named in the availability statement; commit and tag times are recorded by the committer, push times by GitHub's activity log, job times by Slurm.

When	Event
10-21	Stage 0 passes (17060165); Stage C submitted 15:28 (17062103–05)
10-22 12:54	Stage C results pulled
10-22 13:13	runbook proposes the rate rule and a cap of 20 (`288b801`)
10-22 13:14	E0a submitted (17072634–37); the last job ends 16:34
10-22 13:50	rate rule and cap adopted in the status notes; simulator cost model fitted to Stage 0/C server logs (`4435c51`)
10-22 17:03	Stage 0 and C data committed (`d0235a7`)
10-22 17:45	first push to GitHub, with the three commits above
10-23 14:32	E0a results committed (`a76f5e5`)
10-23 14:42	E0b submitted (17084750–55); jobs end by 16:09
10-24 03:13	validation of the calibrated simulator committed (`813cfc1`)
10-24 03:23	prediction spec and code committed before any prediction run (`630ab9e`); on GitHub 03:24
10-24 03:49	predictions B1–B4 committed (`64bc4e3`)
10-24 04:05	independent verification recorded (`877b936`)
10-24 14:52	tag `r6-predictions-frozen` (tagger time)
10-24 14:53	B1–B4 and the verification reach GitHub
10-24 15:06	E0b results pulled
10-24 15:09	E0b data and registration committed in the same second (`cfbc702`, `d279b7b`); registration on GitHub 15:09:54
10-24 15:13	E2 submitted (17093387–404)
10-24 15:38 to 10-25 02:36	E2 jobs run
10-25 02:28	native-pool Stage C and E0a start (17096856–58)
10-25 14:40	registered E2 verdict computed (data `49fbc7b`)
10-25 14:42	native Stage C curve analysed on the cluster; native curve and λ_{miss} in the status notes at 14:47 (`6793fb7`)
10-25 14:45	native E0b (17100938–39) and vLLM E0a (17100940) start
10-25 14:49	native prediction spec committed (`29728e3`)
10-25 14:52	native Stage C and E0a data committed (`fa265c2`)
10-25 14:54	B1n–B4n run on the clean code commit `0336c91`
10-25 15:06	B1n–B4n committed (`e3ea48f`); on GitHub 15:08:28
10-25 15:08	tag `r8-native-predictions-frozen` (tagger time); native rate rule in the status notes (`6cb43e9`)
10-25 17:26	native E0b jobs end; pulled 17:30
10-25 17:31	native E0b data and native registration committed (`f3f55ab`, `b8c19ec`); registration on GitHub 17:31:56
10-25 17:42 to 10-26 01:11	native E2 jobs (17102173–90)
10-25 18:06	vLLM calibration and exploratory registration committed (`65b65e2`, `a78ce07`), on GitHub 18:06:33; vLLM BURST submitted 18:07 (17102318–23)
10-25 22:42 to 10-26 03:24	vLLM BURST jobs
10-26 10:07	native E2 and vLLM BURST verdicts computed (data `f116cf3`)

Table 7: Order of the vLLM events on 2026-10-30 (times CDT, recorded as in Table 6).

When	Event
12:10	vLLM Stage C (17162959–61) and E0b (17162962–63) submitted; 17162959–60 start 12:11
12:16	rule for the vLLM cap and rates in the status notes, on GitHub (`836063a`)
12:31	B1v–B4v committed (`cc01dbf`); tag `r9-vllm-predictions-frozen` 12:32 (tagger time)
12:32	B1v–B4v reach GitHub (12:32:44)
14:36	first vLLM jobs end (Stage C, 17162959–60)
15:35	vLLM Stage C curve analysed on the cluster
16:23	bracket of the five finished vLLM E0b runs (16:23:19); vLLM registration committed (`5a6fbcd`, 16:23:59)
16:24	paired test submitted (17166121–38, 16:24:36); first job starts 16:25:42, second 16:26:17
16:26	vLLM registration and Stage C data (`276c5f0`) reach GitHub (16:26:35)
17:17	vLLM E0b job 17162963, with the 3.0 λ_{ref} run, ends
17:28	vLLM E0b data committed and pushed (`582c2a6`)

notes, which mention E0a, so it is independent but not blind; it also reused the prediction agents' copy of the repository instead of a fresh one, as its record states. Its evidence is in `provenance/codex_prediction_verification/`. Two conditions of the verification failed and are disclosed: the strict containment of B3 passes within the B2 all-stable range, and the platform-specific test failures of the full suite.

In the second round, B2n (native E0b) and B3n (native E2) reached GitHub while native E0b was still running, before any of its runs had finished, and before the native registration or any native E2 job existed. B3n is therefore blind in time. B2n reached GitHub 23 min after the two native E0b jobs were submitted, when both were running, so it is blind in the sense that no result existed, not in submission order. B1n and B4n are not: native Stage C and E0a had finished, and their headline numbers were in the project status notes before the round's specification was written. That specification is shared by all four, so B2n and B3n are blind to the native E0b and E2 outcomes but were specified after the native curve was known, and B3n's cap of 64 is B4n's output. The LLM agent that ran the round worked in a separate copy of the repository at a commit that predates the native data (`fb3510e`) and, by its own record, read nothing under the real-engine results directories; that record has not been checked independently. No independent verifier reran the second round.

In the third round (`notes/PREDICTIONS_vllm.md`), an LLM agent in a separate copy of the repository at `a4bab1d`, which holds the exploratory vLLM BURST data but no vLLM closed curve or edge, ran the calibrated simulator with its token-proportional costs scaled by one factor per batching mode (1.0882 mixed, 1.0806 separate), chosen so that its all-miss token rate equals vLLM's measured 10,349 tokens/s; since the factor is fitted, B1v is not a prediction. By its own

record, which has not been checked independently, it read nothing under the real-engine results directories; the two real vLLM numbers it used, λ_{miss} and the all-miss token rate, were given in its task. The predictions (`cc01dbf`, committed 12:31:58) reached GitHub at 12:32:44, 21 min 48 s after the five vLLM Stage C and E0b jobs were submitted (Table 7); two of these jobs had been running since 12:11 and none ended before 14:36:50. So B2v and B4v are blind in the sense that no result existed when they were pushed, not in submission order; B3v predates the vLLM registration and the paired test. The orchestrating Claude Code session that specified the round knew the outcome of the exploratory vLLM BURST runs, which were in the repository by then, and chose the candidate rates, so the BURST-arm predictions at 1.35 and 1.65 λ_{ref} are blind at most on the predicting agent's side. The notes' remark that their branch was never pushed predates the push. No independent verifier reran the third round. The vLLM registration's own timing is in Appendix B.

D Provenance

Every number in the paper is listed with its source file and commit or job in `paper/NUMBERS.md`. Table 8 gives the data directory and commit behind each result. The 103 commits made before 17:45 CDT on 2026-10-22, including `288b801`, `4435c51` and `d0235a7` cited here, predate a history rewrite on that day that removed the AI co-author trailers from their commit messages and changed nothing else: trees, authors, dates and parent relations are unchanged. `provenance/commit-map.tsv` maps their old hashes, which notes and run records written before the rewrite cite, to the current ones; the repository the old hashes belong to is not on GitHub, so they can be checked only against this map. Their earliest server record is the repository's first push to GitHub, at 17:45:50 CDT on 2026-10-22, with which 97 of them arrived, these three among them, before E0b was submitted on 2026-10-23; one more (`fc870c2`) arrived 2 s later, and five are reachable only from archive tags, which have no push record.

Derived numbers. Quantities that no committed analysis reports directly are computed from committed run data by `figures/paper/derived_numbers.py` and written to `paper/derived/`: the CAPPED drain times; the BURST backlog after the burst and over the window; the E0b hit fractions and realized arrival rates; the arm counts and the post-hoc sign test that counts each seed once; per BURST run, the hit fraction before the burst and the first 100 s window from which no turn hit; and, per run of every arm, the client-side cost (gate waits, turn latency, unfinished sessions) and the realized arrival and completion rates over the window. They cover E0b, E2, native E0b, native E2, vLLM E0b and the exploratory vLLM BURST runs. Each derived file names a run both by its path on the cluster and by its path from the repository root.

Table 8: Data behind each result (paths relative to the repository root at tag `arxiv-v1`).

Result	Directory (commit)
Stage 0, C	`results/nova/s0_smoke_17060165_pulled_20261021-152347/`, `results/nova/stageC_1706210x_pulled_20261022-125454/` (`d0235a7`)
E0a	`results/nova/e0a_1707263x_pulled_20261023-142037/` (`a76f5e5`)
E0b	`results/nova/e0b_1708475x_pulled_20261024-150639/` (`cfbc702`)
E2	`results/nova/e2_170933xx_pulled_20261025-144558/` (`49fbc7b`)
native, grid, vLLM	`results/nova/r7_native_grid_vllm_pulled_20261025-145126/` (`fa265c2`)
native E0b	`results/nova/e0b_native_1710093x_pulled_20261025-173038/` (`f3f55ab`)
vLLM E0a	`results/nova/e0a_vllm_17100940_pulled_20261025-180814/` (`b273241`)
native E2, vLLM BURST	`results/nova/e2_native_vllm_burst_pulled_20261026-101256/` (`f116cf3`)
vLLM C	`results/nova/stageC_vllm_1716295x_pulled_20261030-153621/` (`276c5f0`)
vLLM E0b	`results/nova/e0b_vllm_1716296x_pulled_20261030-172810/` (`582c2a6`)
validation	`results/r6_validate_stagec/20261024-025222/` (`813cfc1`)
B1–B4	`results/r6_predict_report/20261024-034707/`, `notes/PREDICTIONS.md` (`64bc4e3`)
B1n–B4n	`results/r8_native_report/20261025-150146/`, `notes/PREDICTIONS_native.md` (`e3ea48f`)
B1v–B4v	`results/r9_vllm_report/20261030-122723/`, `notes/PREDICTIONS_vllm.md` (`cc01dbf`)
robustness	`results/r7_report/20261024-233554/` (`c653498`); `results/r7_traces_sim/r7_report/20261025-012451/`, `results/r7_traces_sim_adj/r7_report/20261025-012453/` (`b530e55`)
mechanisms	`results/prefix_meta_baselines_audit_summary/20261025-001850/` (`dc76032`); `results/prefix_meta_r4_curve/20261021-050856/`, `results/prefix_meta_r4/20261021-051550/` (`a1d485b`); `results/prefix_meta_mech/20261021-025902/` (`18ff187`)
push log, jobs	`provenance/`
figures	`figures/paper/`, `paper/figs/README.md`
derived	`paper/derived/`, `figures/paper/derived_numbers.py`

Notes that support the text. The math check of the model, `notes/MATHCHECK_thrashing_curve.md`, was done by an LLM agent on 2026-10-21 and has not been independently reviewed; it predates the present assumptions (A1)–(A4) and Lemma 1, and where it differs from §3 and Appendix E, those hold; the verification of Stage C (three-source hit check, eviction survival times, prompt continuity) is `notes/STAGEC_VERIFICATION.md`; the audit of the PEEK, Continuum and CONCUR rules against released code is `notes/BASELINES_AUDIT.md`; an audit of the citations as of 2026-10-25, the scoop check and a first reading of the SGLang and vLLM sources are in `notes/lit/CITATION_AUDIT_2026-10-25.md` (that reading went twice through a page summariser, a web tool that returns a language model's summary of each file, without verbatim code; the line references of §2 and Appendix E were later read verbatim in the files at the tags), and that audit predates the citations added since, among them Yuan et al. [75], which it lists as not cited; the trace statistics are in `notes/lit/AGENT_TRACES.md`. Where the research log and a committed CSV differ in the last digit (three hit fractions and one backlog of the policy grid), the paper uses the CSV.

E Proofs

These proofs were checked by AI review agents (see the statement on the use of AI tools) and have not been refereed.

Model and conventions. In the closed system of §3.1, n sessions are present and a finished session is replaced at once by a fresh one, whose first request joins the queue. At time 0 every session is fresh, so no tool call is in progress, as in the synchronized start of our closed runs. A request joins the queue when its session sends it, waits, and is admitted; it leaves the queue only by admission. An admitted request runs until it completes or, if the engine preempts it, rejoins the queue. At a common instant joins are processed before admissions, as when an engine receives requests before each scheduling pass, and requests that join at the same instant enter in some order; the requests waiting when request i joins are those in the queue just before it joins and those that join at the same instant ahead of it. When a request that is not its session's last completes, the session starts a tool call, and when the call ends the session's next request, a follow-up, joins the queue. By (A1) evicted KV is discarded, not offloaded. A follow-up's wait runs from its joining the queue to its first admission, and its reuse, and whether it hits, are counted at that admission. SGLang logged no retraction in any of the 73 server logs of our SGLang runs. When a running request cannot get a block for its next tokens, vLLM preempts the most recently admitted running request and returns it to the head of its queue (*Other orders* below); its closed runs logged 12 to 72 preemptions per run from $n = 32$ on (72 at $n = 300$, in a run that completed 5,395 requests), and its cache-disabled reference 9 or 10 per 32-session run. Since w_n and C count each prompt token once, at its request's first admission, recomputation after a preemption enters neither, and the proofs below go through with re-admissions counted among the admissions that (A2) bounds. By (A1) each session is at every moment in exactly one of three states: its request waits, its request runs, or it is in a tool call. Tool times have law F and mean $\zeta < \infty$. A long-run rate is $\lim_{t\to\infty} A(t)/t$ for a count or sum $A(t)$ over $[0, t]$; we assume that these limits exist almost surely and are deterministic, as for a positive Harris recurrent Markov process from any initial state, and that λ_{miss} and C, the limits of $\mu(n)$ and w_n as $n \to \infty$ with caching disabled, exist. Every $A(t)/t$ used below is bounded for fixed n and $t \geq 1$, since at most $\alpha t + \beta$ requests are admitted by time t, so by bounded convergence each rate also equals $\lim_{t\to\infty} E[A(t)]/t$, and bounds on expectations carry over to rates.

Lemma 1 (short waits are rare)**.** *Assume (A1) and (A2), with Q_0 and S_0 as in (A2) (both 0 for request-level FCFS). Let $\gamma_n(K)$ be the long-run rate of follow-ups that wait less than K, and let $\kappa = \lfloor \alpha + \beta \rfloor + B$, $y = \kappa(1 + \zeta)$ and $m = n - B - \alpha K - \beta - Q_0 - S_0$. If $m > \kappa$, then*

$$\gamma_n(K) \leq \frac{y + y^2 + \kappa y}{m - \kappa},$$

so $\gamma_n(K) = O(1/n)$ for every fixed $K > 0$.

The general ordering condition of (A2), the only property of the order that the proof uses, is this: if more than Q_0 requests wait at every moment from a follow-up i's joining the queue to its first admission, counting at a moment with admissions the requests that wait just before them, then all but at most S_0 of the requests that were waiting when i joined are admitted, for the first time or again, no later than that admission. Request-level FCFS meets it with $Q_0 = S_0 = 0$, since i itself waits.

Proof. For a follow-up i write q_i for the time it joins the queue, D_i for its wait and Q_i for the number of requests waiting when it joins, and let $X(t)$ be the number of sessions in tool calls at time t.

A short wait needs a short queue. Let $D_i < K$. At most $\alpha K + \beta$ admissions occur in $[q_i, q_i + D_i]$, i's own included, so fewer than $\alpha K + \beta$ occur before i's. If $Q_i \geq \alpha K + \beta + Q_0 + S_0$, then at every moment of $[q_i, q_i + D_i]$, counted as above, more than Q_0 of the Q_i requests still wait, because requests leave the queue only by admission; so at least $Q_i - S_0$ of them are admitted no later than i, and $Q_i - S_0 + 1 \leq \alpha K + \beta$, a contradiction. Hence $Q_i < \alpha K + \beta + Q_0 + S_0$.

A short queue needs almost every session in a tool call. Just before q_i each of the other $n - 1$ sessions is waiting (at most Q_i of them), running or in a tool call, and i's own session is in a tool call or, if that call has length 0, running; at most B sessions run. So $X(q_i-) \geq n - Q_i - B > m$ whenever $D_i < K$.

From the instants at which tool calls end to fixed times. Follow-ups join the queue when tool calls end, which happens more often when many sessions are in tool calls, so an average over these instants is not a time average; we avoid it as follows. A request that completes in an interval $(s, s+1]$ was admitted, for the last time, in that interval or was running at time s, so at most κ requests complete, and at most κ tool calls start, in

any such interval. Fix an integer $j \geq 0$. A follow-up that joins the queue in $(j, j+1]$ ends a tool call that was in progress at j or started in the interval, so at most $X(j) + \kappa$ follow-ups join in it, and $X(q-) \leq X(j) + \kappa$ for every $q \in (j, j+1]$. By the previous step, the number of follow-ups that join in $(j, j+1]$ and wait less than K is at most $g(X(j))$, where $g(x) = (x + \kappa)\,\mathbf{1}\{x > m - \kappa\}$.

A bound on $X(t)$ that does not depend on n. Fix t and split $(-\infty, t]$ into the intervals $I_k = (t-k-1,\ t-k]$, $k \geq 0$; no tool call starts before time 0, and at most κ start in each I_k. Let $(Z_{kl})_{k\geq 0,\, 1\leq l\leq \kappa}$ be i.i.d. with law F, independent of everything else. Give the l-th call that starts in I_k the duration Z_{kl}, and calls that start after t fresh durations. This does not change the law of the system: by (A1) each call's duration is independent of all other randomness, and the system's evolution up to any time depends on a call's duration only through whether the call has ended by then; the index (k, l) of a call is fixed when the call starts, so the variable it receives has not been used and is independent of the past at that moment, and the two constructions agree in law by induction over successive events. A call that started in I_k and is still in progress at t has a duration greater than k, so in this construction

$$X(t) \leq Y = \sum_{k\geq 0} \sum_{l=1}^{\kappa} \mathbf{1}\{Z_{kl} > k\},$$

whose law depends on neither t nor n. Y is a sum of independent indicators with $E[Y] = \kappa \sum_{k\geq 0} P(Z > k) \leq \kappa(1+\zeta) = y$, so $\operatorname{Var} Y \leq E[Y]$ and $E[Y^2] \leq y + y^2$.

Conclusion. Since g is non-decreasing and $\mathbf{1}\{Y > m-\kappa\} \leq Y/(m-\kappa)$,

$$E[g(X(j))] \leq E[g(Y)] \leq \frac{E[Y^2] + \kappa E[Y]}{m - \kappa} \leq \frac{y + y^2 + \kappa y}{m - \kappa}.$$

Summing over the unit intervals of $(0, t]$, for integer t, bounds the expected number of follow-ups that join by time t and wait less than K by t times this quantity; dividing by t and letting $t \to \infty$ gives the claim. □

The bound is of order $1/n$, but its constants grow with the admission rate and the mean tool time, so it is a statement about the limit, not about the populations we ran.

Proof of Proposition 1. Reuse needs waiting token-time. Let follow-up i of session s join the queue at q_i, wait D_i, and reuse r_i prompt tokens from cache at its first admission. By (A1) these tokens are s's own, computed by its earlier requests, and evicted KV is discarded, so each of them was in the pool throughout $[q_i, q_i + D_i]$, during which s has no running request to recompute it. These pairs of token and time are distinct for different requests: sessions share no tokens, and the waits of one session's requests do not overlap. At most M tokens are resident at any time, so for every t, $\sum_i r_i D_i \leq Mt$ over the follow-ups first admitted by t. This step uses neither the order nor the eviction rule, so it holds for clairvoyant rules too.

Reuse vanishes. Fix $K > 0$. By time t the follow-ups that waited at least K reused at most Mt/K tokens, and each of the others at most $L_{\max} = 40{,}350$, the longest prompt. So the long-run rate ρ_n of cache-served prompt tokens satisfies $\rho_n \leq M/K + L_{\max}\gamma_n(K)$, and Lemma 1, with the order's Q_0 and S_0, gives $\limsup_{n\to\infty} \rho_n \leq M/K$ for every K, so $\rho_n \to 0$. A hit reuses more than half its prompt, hence more than $L_0/2$ tokens, so hits occur at a rate of at most $2\rho_n/L_0 \to 0$.

Work. A session with T turns sends $W(T)$ prompt tokens (§3.1), and each, counted once at its request's first admission, is either served from cache or computed. The closed system has n slots, each holding one session at a time and a fresh one after it finishes, so each slot holds a sequence of sessions with i.i.d. turn counts, and by the strong law of large numbers completed sessions bring prompt tokens at the long-run rate $\mu(n)E[W]$ and turns at the rate $\mu(n)E[T]$; sessions in progress hold at most $nW(60)$ tokens and $60n$ turns and do not affect rates. Hence $w_n = \mu(n)E[W] - \rho_n$. With caching disabled nothing is reused, so the same count gives $C = \lambda_{\text{miss}} E[W]$. So $\mu(n) = (w_n + \rho_n)/E[W]$, and (A3) gives $\limsup_{n\to\infty} \mu(n) \leq C/E[W] = \lambda_{\text{miss}}$. The hit fraction is the rate of hits over the rate of turns, so it is at most $2\rho_n/(L_0\,\mu(n)E[T])$ and tends to 0 whenever $\liminf_{n\to\infty} \mu(n) > 0$. Under (A4), $\mu(n) \to \lambda_{\text{miss}} > 0$. □

Other orders. The proof uses the order only through Lemma 1, so it holds for any order that meets the condition above for some Q_0 and S_0. In SGLang 0.5.22 the admission loop walks the waiting queue in order and stops at the first request that cannot be added or when the batch is full (`python/sglang/srt/managers/scheduler.py`, lines 4080–4176 at tag v0.5.22); it skips a request only for LoRA, pooling-mode or storage-prefetch conditions that our runs do not use. New requests join the queue at its tail (line 3366), as would a retracted request (lines 4369–4416). FCFS sorts the queue only under priority scheduling, which none of our runs enabled (`python/sglang/srt/managers/schedule_policy.py`, lines 227–231), and LPM sorts it only while at most 128 requests wait (lines 133 and 300–311); with more waiting, LPM takes the FCFS branch, which leaves the queue's order unchanged (lines 260–261), so new requests queue behind all waiting ones. So FCFS meets the condition with $Q_0 = S_0 = 0$ and stock LPM with $Q_0 = 128$ and $S_0 = 0$. vLLM 0.32.0 admits from the head of its FCFS queue and stops at the first request whose allocation fails (`vllm/v1/core/sched/scheduler.py`, lines 1275–1282 at tag v0.32.0), and new requests join at the tail (`vllm/v1/core/sched/request_queue.py`, lines 75–94). It also preempts the most recently admitted running request when a running request cannot get a block (`vllm/v1/core/sched/scheduler.py`, lines 794–844), frees its blocks and returns it to the head of the queue (lines 1571–1613). The condition concerns first admissions, so this re-admission ahead of earlier requests does not violate it, and the re-admissions count among the admissions that (A2) bounds. With asynchronous scheduling, vLLM also skips a preempted request until its output still in flight drains, within the pipeline depth (lines 933–939), and a follow-up admitted meanwhile passes it; only requests preempted in the last few steps are skipped

this way, at most a fixed multiple of the running limit, so S_0 stays bounded. vLLM also skips requests that are blocked, for example waiting for a KV transfer or over a LoRA limit, which our runs do not involve. Its FCFS thus meets the condition with $Q_0 = 0$ and a bounded S_0. The line numbers refer to the files at these tags, read verbatim, not through the page summariser of Appendix D. Program-level FCFS meets the condition for no Q_0 and S_0: a follow-up of an old session is admitted before requests of newer sessions that were waiting when it joined, however long the queue.

The fluid statements of §3.3. With μ continuous and piecewise linear, Eq. (1) has a unique solution from every $x_0 \geq 0$; it is monotone in time, and it converges to a fixed point or diverges. Let $0 < \lambda < \mu^*$. Then $\mu(n) = \lambda$ has exactly one solution n_1 in $(0, n^*)$, because μ increases strictly from 0 to μ^* there, and at most one in (n^*, ∞), because μ decreases strictly there. If $\lambda < \mu_\infty$, then $\mu > \mu_\infty > \lambda$ beyond n^*, so $\mu < \lambda$ on $[0, n_1)$ and $\mu > \lambda$ on (n_1, ∞), and every solution converges to n_1. If $\mu_\infty < \lambda$, the second solution n_2 exists, because $\mu(n^*) > \lambda$ and $\mu \to \mu_\infty < \lambda$; then $\mu > \lambda$ on (n_1, n_2) and $\mu < \lambda$ on (n_2, ∞), so n_2 is unstable, and for $x_0 > n_2$, $\dot{n} \geq \lambda - \mu(x_0) > 0$ along the solution and $n \to \infty$. If $\lambda > \mu^*$, then $\mu < \lambda$ everywhere and every solution diverges. A solution from $x_0 > n_1$ converges to n_1 if and only if $\mu > \lambda$ on all of $(n_1, x_0]$: if so, it decreases and stays in $(n_1, x_0]$, where n_1 is the only possible limit; if $\mu(x) \leq \lambda$ for some $x \in (n_1, x_0]$, it never falls below x, where the drift is non-negative. For $x_0 > n^*$, $\mu > \lambda$ on $(n_1, n^*]$ and the minimum of μ on $[n^*, x_0]$ is $\mu(x_0)$, so the condition is $\lambda < \mu(x_0)$, and $\mu(x_0)$ decreases to μ_∞ as $x_0 \to \infty$. Without the monotone tail the same argument gives the condition $\lambda < \min_{[n^*, x_0]} \mu$, which tends to $\inf_{n \geq n^*} \mu(n)$, and a point beyond n^* where μ crosses λ from below is a stable congested fixed point. Under the assumptions of Proposition 1 this infimum is at most $\liminf_{n \to \infty} \mu(n) \leq \lambda_{\text{miss}}$, and it equals λ_{miss} if, in addition, $\mu(n) \geq \lambda_{\text{miss}}$ for every $n \geq n^*$.